G-Stride

PRODUCT DOCUMENTATION · CURRENT VERIFIED BUILD

Product Implementation, Backlog and Defect Status Report

A detailed evidence-based account of implemented functionality, every user story and acceptance criterion, remediated defects, remaining product gaps, and workspace-level capability.

DOCUMENT **Q4H-PROD-STATUS-2.0**	PRODUCT BASELINE **G-Stride 2.1.0 development line; v2.0.0 remains frozen**
PREPARED **30 July 2026**	CLASSIFICATION **Internal product and training documentation**

Screens containing configured target or account information have been anonymized. Passwords and credential values are never included.

DOCUMENT MAP

Contents

Interpretation rule. “Implemented” means the tracker criterion is covered by repository evidence. “Partial” identifies delivered capability that still has acceptance work. “Open” is not yet accepted. “Deferred” is intentionally outside the current release scope.

CURRENT PRODUCT POSITION

1 Executive implementation summary

G-Stride is a working, Canvas First SAP test-automation product spanning reusable authoring, controlled execution, immutable evidence and operational quality reporting.

50 **Backlog items**	**45** **Implemented**	**156/173** **Accepted criteria**	**234** **Catalogue tests**
0 **Partial items**	**2** **Not started**	**3** **Deferred**	**59/59** **Defects remediated**

Delivered product spine

- Owner identity, encrypted SAP credential handling and authoritative target classification.
- Routeable Tests, datasets, Business Processes, Regression Packs, runs and audit records.
- Typed visual authoring from Single Test contracts through sequential hand-offs and independent Packs.
- Server-owned preflight, immutable effective-data snapshots, controlled execution and safe reruns.
- One canonical evidence PDF referenced by both Execution Center and Audit & Evidence.
- Accessible responsive shell, NVDA verification and release-governance evidence.

Current quality position

- Latest final source verification: 57 core, 44 isolated API and 40 isolated UI checks passed.
- Eight live-SAP checks remained deliberately gated in the final source verification.
- High-confidence secret scan passed across 259 non-ignored repository files.
- 20 recorded quality runs are retained in the current quality history.
- All 59 recorded historical failures have a remediation run; no open ledger defect remains.

Important distinction. The absence of an open recorded defect does not mean the backlog is complete. 5 items still contain partial, open or deferred acceptance scope and are listed in Section 5.

FUNCTIONAL INVENTORY

2 Detailed features by workspace

The following inventory describes current behavior rather than the original navigation labels. Business Processes and Regression Packs share one “Processes & Packs” workspace.

Automation Overview

A task-focused landing canvas for status, value, recent work and quick entry.

- Live counts for Tests, Business Processes, controls and evidence records.
- Recorded execution totals, pass/fail outcomes and automated runtime.
- Transparent modeled manual effort, potential time saved and cost-saved scenarios.
- Recent work cards and direct Create Test / New Execution actions.
- Authoritative configured-target state displayed in the shared header.

Remaining consideration: Exact recent-item deep links, readiness alerts, saved cost preferences and accessible historical trends remain.

Control Object Repository

Capture, curate and reuse SAP UI controls instead of duplicating selectors in Tests.

- Live-screen scan starts from the configured SAP target.
- Bulk scan and continuous Ctrl/Cmd interactive picking with duplicate detection.
- Domain/App ID browsing, local filtering, stable object routes and live highlight.
- Object edit, delete and keyboard-accessible ordering.
- Explicit App ID confirmation and curation before saving.

Remaining consideration: Persistent selector health/history, dependency-aware changes and contextual return to Compose remain.

Compose

Find, create, edit, validate and publish reusable Single Tests.

- Routeable Test Library with name, process-area, application and readiness filters.
- Guided blank-or-template creation with conflict-safe persistence.
- Visual typed Test contracts with input/output types and sensitivity.
- Step values distinguish literals, datasets, system context and prior outputs.
- Schema-driven ModuleCall editing, reusable object selection and keyboard reordering.
- Publish blocking for unresolved values, missing parameters/objects and output collisions.

Remaining consideration: No outstanding BL-020/BL-021 acceptance gap after the current reconciliation.

Test Data

Author reusable scalar, tabular, nested and relational transaction data.

- Flat CSV creation/editing with rich RFC4180 round-trip and dirty-state protection.
- Nested JSON transaction authoring for one or many headers with owned line items.
- Relational CSV builder with header/child keys and owned collection preview.
- Validation for duplicate keys, orphan children, missing keys and name collisions.
- Effective-data preview and immutable execution snapshots through preflight.

Remaining consideration: The full searchable library, typed/sensitive columns and dependency-safe rename/removal remain.

Processes & Packs

Model sequential business flows and independent regression collections visually.

- Business Processes sequence Tests and carry typed outputs into later stage inputs.
- Visual stage ordering, lifecycle, domain, application and data bindings.
- Validation for unresolved inputs, cycles, forward references, unknown/duplicate outputs and type mismatch.
- Regression Packs contain independent Test or Business Process members.
- Member-specific data, application, session and failure policies with draft/published lifecycle.
- Stable routes and direct execution of published mixed Saved Packs.

Remaining consideration: No outstanding BL-028/BL-029 acceptance gap after the current reconciliation.

Execution Center

Prepare, preflight, approve, run, monitor and safely rerun controlled automation.

- Execution types: Single Test, Business Process, Pack · Tests, Pack · Processes and Saved Pack.
- Four-stage flow: Scope, Data & policies, Preflight and Review.
- Server-owned preflight validates target, credentials, artifacts, versions, data, contracts and policies.
- Exact effective-data preview, mapping review, plan hash and immutable snapshot.
- Determinate progress for known work, persisted run routes and reconnect recovery.
- Fail-stop transaction behavior retains earlier evidence; no automatic reversal.
- Rerun difference review, immutable lineage and failed-scope safety restrictions.

Remaining consideration: Complete six-level event persistence/reconciliation and exact terminal-failure correction links remain.

Audit & Evidence

Search immutable execution history and open one canonical evidence record per run.

- Flat newest-first run library rather than date-folder navigation.
- Search and client filters for outcome, mode and date.
- Stable audit detail routes with executor, target and source context.
- Canonical evidence PDF shared with Execution Center—no duplicate evidence document.
- Evidence branding, Test name headers, redaction and retention governance.
- Preserved rerun lineage and immutable execution identifiers.

Remaining consideration: Server-side multi-field filtering, paging/sorting and full parent/child lineage traversal remain.

Settings, identity and account menu

Control owner identity, appearance, help and test-system connections.

- Single preserved workspace-owner identity with Google registration/sign-in support.
- Profile menu provides Settings, Help and Sign out.
- Appearance selection is managed inside Settings.
- SAP connection captures target URL and credentials without returning passwords to the browser.
- OS credential vault where available and encrypted local fallback for detached Windows sessions.
- Salesforce, Oracle and ServiceNow connection panels retained as future configuration surfaces.

Remaining consideration: Multi-user roles remain intentionally deferred; only SAP is integrated with execution and live scan.

G-Stride Test Operations companion app

Provide release-quality visibility across the automated product test estate.

- Repository-generated 149-test catalogue grouped by feature and workspace.
- Latest status, execution mode and recorded duration per test.
- Historical execution dashboard backed by recorded runs.
- Persistent failure ledger including failures later remediated and passed.

Remaining consideration: It is an operational companion rather than a primary Studio workspace.

41-ITEM SOURCE OF TRUTH

3 Granular product backlog and acceptance status

Every consolidated backlog item is shown with its description, user story, release/priority metadata, criterion-by-criterion status, implementation coverage and evidence.

Status	Items	Meaning
Implemented	45	All recorded acceptance criteria are satisfied.
Partial	0	Some functionality exists; one or more criteria still need delivery.
Not started	2	No acceptance criterion is complete.
Deferred	3	Deliberately outside the present release scope.

BL-001 · PB-001 · IMM-01 Implemented

Show verified execution context

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T0 · Delivered baseline**	**Not yet delivered**	**Shell, Execution Center**

ACCEPTANCE
3/3

Description

Display only server-backed workspace and target context and avoid unsupported connection claims.

User story

As a tester, I want verified target context so that I know where an execution will act.

Acceptance criteria

1	Shell and review use server-backed target metadata.	Implemented
2	Unavailable or stale context is explicit and blocks unsafe execution.	Implemented
3	No decorative control implies a successful connection.	Implemented

Implementation coverage

Unsupported environment claims were removed; server-backed SAP configuration is displayed. Freshness verification remains strengthened by BL-014.

Evidence: App.tsx target label · P0_IMPLEMENTATION_REPORT.md
Last reconciled: 2026-07-27

BL-002 · PB-002 · IMM-02 Implemented

Review execution impact before Start

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T0 · Delivered baseline**	**Not yet delivered**	**Execution Center**

ACCEPTANCE
3/3

Description

Present target, scope, data, side effects and evidence behavior before starting execution.

User story

As an operator, I want an impact review so that I can prevent unintended SAP changes.

Acceptance criteria

1	Review shows target, mode, ordered scope, data and browser policy.	Implemented
2	SAP side effects and cancellation boundary are explicit.	Implemented
3	Duplicate submission produces one run ID.	Implemented

Implementation coverage

Review, warning acknowledgement and duplicate-submit protection are implemented; authoritative matrix enhancements continue in BL-030.

Evidence: RunPanel.tsx · regression/ui/run-tab.test.js
Last reconciled: 2026-07-27

BL-003 · PB-003 · IMM-03 Implemented

Use truthful dashboard states

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T0 · Delivered baseline**	**Not yet delivered**	**Automation Overview**

ACCEPTANCE
3/3

Description

Source Overview values from APIs and distinguish loading, empty and unavailable data.

User story

As a test manager, I want trustworthy dashboard states so that I do not act on fabricated information.

Acceptance criteria

1	Metrics and recent records come from APIs.	Implemented
2	Loading, empty and unavailable states are distinct.	Implemented
3	No fallback execution or document record is fabricated.	Implemented

Implementation coverage

Workspace counts, immutable runs and evidence are API-backed with retry and empty states.

Evidence: AutomationOverview.tsx · regression/ui/overview.test.js
Last reconciled: 2026-07-27

BL-004 · PB-004 · IMM-09 Implemented

Restrict local Studio exposure

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T0 · Delivered baseline**	**Not yet delivered**	**Platform**

ACCEPTANCE
3/3

Description

Bind locally by default and require an explicit decision before network exposure.

User story

As the workspace owner, I want Studio locally restricted so that unfinished controls are not exposed.

Acceptance criteria

1	Default host is 127.0.0.1.	Implemented
2	Non-loopback binding requires an explicit override.	Implemented
3	Startup communicates network exposure risk.	Implemented

Implementation coverage

Loopback is the default and host override is explicit.

Evidence: packages/studio-server/src/standalone.ts · P0_IMPLEMENTATION_REPORT.md
Last reconciled: 2026-07-27

BL-005 · PB-005 · IMM-04 Implemented

Restore functional workspace styling

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	T0 · Delivered baseline	Not yet delivered	All workspaces

ACCEPTANCE
3/3

Description

Provide coherent forms, tables, statuses, dialogs, drawers and responsive functional layouts.

User story

As a user, I want consistent functional styling so that every workspace is readable and operable.

Acceptance criteria

1	Functional classes have visible states and spacing.	Implemented
2	Status and feedback are not color-only.	Implemented
3	Core screens remain usable at supported widths.	Implemented

Implementation coverage

Canvas First shell and functional compatibility styles are present; component-system consolidation remains BL-011.

Evidence: packages/studio-web/src/index.css · P0_IMPLEMENTATION_REPORT.md
Last reconciled: 2026-07-27

BL-006 · PB-006 · IMM-05 Implemented

Preserve correct UI encoding

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T0 · Delivered baseline**	**Not yet delivered**	**All workspaces**

ACCEPTANCE
3/3

Description

Keep source content UTF-8 and prevent accidental character corruption.

User story

As a user, I want readable product copy so that technical symbols and punctuation are unambiguous.

Acceptance criteria

1	Source is valid UTF-8.	Implemented
2	Build and browser output preserve characters.	Implemented
3	Display-tool mojibake is not written back to source.	Implemented

Implementation coverage

Source verification closed the issue; PowerShell display encoding was identified as the false signal.

Evidence: PRODUCT_BACKLOG.md P0 status
Last reconciled: 2026-07-27

BL-007 · PB-007 · IMM-07 Implemented

Protect unsaved artifact work

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	T0 · Delivered baseline	Not yet delivered	Compose, Test Data, Processes & Packs, Execution Center

ACCEPTANCE
3/3

Description

Guard dirty tests, datasets, processes and execution drafts during navigation and reload.

User story

As an author, I want unsaved changes protected so that navigation does not destroy my work.

Acceptance criteria

1	Dirty state reaches shell navigation.	Implemented
2	Artifact switching and browser unload require confirmation.	Implemented
3	New drafts are dirty from their first meaningful change.	Implemented

Implementation coverage

Composer, Dataset, Group and Execution draft guards are implemented and regression tested.

Evidence: App.tsx navigation guard · regression/ui/unsaved-guards.test.js
Last reconciled: 2026-07-27

BL-008 · PB-008 · IMM-06 Implemented

Provide baseline keyboard and screen-reader support

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T0 · Delivered baseline**	**Not yet delivered**	**All workspaces**

ACCEPTANCE
3/3

Description

Make core shell navigation, forms, actions and feedback semantically operable.

User story

As a keyboard or screen-reader user, I want equivalent access to core Studio actions.

Acceptance criteria

1	Primary navigation and actions use semantic controls.	Implemented
2	Focus is visible and labels are programmatic.	Implemented
3	Errors and status changes are announced where required.	Implemented

Implementation coverage

P0 semantic baseline is present; complex-widget behavior and full accessibility/release verification were subsequently closed by BL-016 and BL-017.

Evidence: App.tsx skip link and headings · P0_IMPLEMENTATION_REPORT.md
Last reconciled: 2026-07-27

BL-009 · PB-009 · IMM-10 Implemented

Run isolated frontend regression tests

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	T0 · Delivered baseline	Not yet delivered	Quality Engineering

ACCEPTANCE
3/3

Description

Exercise UI workflows against temporary synthetic fixtures without live SAP side effects.

User story

As a developer, I want isolated UI regression coverage so that changes can be verified safely.

Acceptance criteria

1	Harness uses temporary storage.	Implemented
2	Execution is disabled unless explicitly approved.	Implemented
3	Fixtures and temporary databases are removed after the run.	Implemented

Implementation coverage

Isolated UI harness covers Overview, Compose, Data, Groups, Execution, Audit and dirty guards.

Evidence: regression/run-isolated-ui.js · regression/ui
Last reconciled: 2026-07-27

BL-010 · PB-010 · IMM-08 Implemented

Recover live monitoring connections

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T0 · Delivered baseline**	**Not yet delivered**	**Execution Center**

ACCEPTANCE
3/3

Description

Reconnect polling safely and preserve run identity after temporary server failures.

User story

As an operator, I want monitoring to recover so that a network interruption does not hide a live run.

Acceptance criteria

1	Polling has bounded backoff and cancellation.	Implemented
2	Last update and connection state are visible.	Implemented
3	Refresh never starts duplicate execution work.	Implemented

Implementation coverage

Recursive polling, backoff, manual retry and cleanup are implemented.

Evidence: RunPanel.tsx · P0_IMPLEMENTATION_REPORT.md
Last reconciled: 2026-07-27

BL-011 · PB-011 · FND-01 · FND-02 Implemented

Consolidate design tokens and workspace primitives

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	T1 · 2.0 Alpha	Not yet delivered	Design System, All workspaces

ACCEPTANCE
3/3

Description

Replace local styling variations with reusable Canvas First layout and interaction primitives.

User story

As a product team, we want shared primitives so that workspaces remain visually and behaviorally consistent.

Acceptance criteria

1	Tokens cover color, type, spacing, focus, status and elevation.	Implemented
2	Page header, toolbar, card, table, drawer and empty-state primitives are reused.	Implemented
3	Light and dark themes meet contrast requirements.	Implemented

Implementation coverage

Canvas First now exposes shared type, spacing, focus, status, elevation and theme tokens plus reusable page header, toolbar, card, table, drawer, empty and async-feedback primitives. Automated checks cover normal-text contrast in both themes.

Evidence: index.css T1.3 Canvas First primitives · WorkspacePrimitives.tsx · DocumentsPanel.tsx primitive adoption · t1-foundations.test.js theme contrast
Last reconciled: 2026-07-29

BL-012 · PB-012 · FND-03 Implemented

Provide stable artifact and run routes

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	T1 · 2.0 Alpha	2.1.0	Shell, All workspaces

ACCEPTANCE
3/3

Description

Give every workspace, artifact and run a recoverable URL with navigation guards.

User story

As a user, I want stable URLs so that refresh, bookmarks and correction links preserve context.

Acceptance criteria

1	Workspace routes survive refresh and browser navigation.	Implemented
2	Tests, datasets, processes, packs and audit records have stable detail routes.	Implemented
3	Dirty-state guards apply to route changes.	Implemented

Implementation coverage

Canonical routes restore Tests, datasets, Business Processes, Regression Packs, object selections, execution runs and audit records across refresh and browser back/forward. Preflight findings open the exact dataset/source or SAP setting, and shell, tab, browser-history and unload navigation protect unsaved work.

Evidence: routes.ts · App.tsx route orchestration · ProcessPacksWorkspace.tsx · PackEditor.tsx · DocumentsPanel audit detail · executionPreflight.ts correctionRoute · stable-routes.test.js · packs-tab.test.js · unsaved-guards.test.js
Last reconciled: 2026-07-30

BL-013 · PB-013 · FND-04 Implemented

Complete the responsive application shell

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	T1 · 2.0 Alpha	Not yet delivered	Shell

ACCEPTANCE
3/3

Description

Keep stable navigation, account actions and workspace context usable across supported widths.

User story

As a user, I want a calm responsive shell so that navigation does not dominate the work canvas.

Acceptance criteria

1	Sidebar collapses without hiding destinations.	Implemented
2	Account menu owns Settings, Help and Sign out.	Implemented
3	Header actions wrap or simplify at narrow widths without overlap.	Implemented

Implementation coverage

The compact sidebar retains programmatic destination names, the account menu owns Settings, Help and Sign out with complete menu-key behavior, and target plus workflow actions reflow without shell overflow at narrow widths.

Evidence: App.tsx responsive shell · AccountMenu.tsx menu keyboard contract · index.css narrow shell rules · t1-foundations.test.js narrow-width journey
Last reconciled: 2026-07-29

BL-014 · PB-014 · CTX-01 Implemented

Provide authoritative workspace and environment context

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T1 · 2.0 Alpha**	**Not yet delivered**	**Shell, Settings, Control Object Repository, Execution Center**

ACCEPTANCE
3/3

Description

Unify owner, SAP target, safety class, profile reference and verification freshness.

User story

As an operator, I want one authoritative context so that every workspace refers to the same target.

Acceptance criteria

1	Server returns non-secret workspace and target context.	Implemented
2	Shell, scan and execution review show the same context and freshness.	Implemented
3	A stale or mismatched context blocks Start with recovery guidance.	Implemented

Implementation coverage

One non-secret server context now supplies owner, SAP hostname, profile, safety class and 24-hour verification freshness to the shell, live scanner and execution review. Preflight hashes that context, blocks unknown/stale targets and invalidates mismatches before Start.

Evidence: executionContext.ts · workspaceGovernance.ts · /api/workspace-context · ObjectScanner.tsx · RunPanel.tsx · execution-preflight.test.js · t1-security-context.test.js
Last reconciled: 2026-07-29

BL-015 · PB-015 · FND-05 Implemented

Standardise async and feedback states

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	T1 · 2.0 Alpha	Not yet delivered	All workspaces

ACCEPTANCE
3/3

Description

Use consistent loading, empty, warning, error, success and retry behavior.

User story

As a user, I want predictable feedback so that I know whether Studio is working, waiting or blocked.

Acceptance criteria

1	Every API-backed surface has loading, empty and error states.	Implemented
2	Errors are actionable and safe.	Implemented
3	Success is associated with the item or action that changed.	Implemented

Implementation coverage

API-backed workspaces now use consistent announced loading, empty, actionable error/retry and item-associated save-success behavior. Existing execution, scan and settings feedback uses the same status semantics.

Evidence: WorkspacePrimitives.tsx AsyncFeedback and EmptyState · TestCaseEditor.tsx · DataEditor.tsx · GroupEditor.tsx · DocumentsPanel.tsx · ObjectBrowser.tsx
Last reconciled: 2026-07-29

BL-016 · PB-016 · FND-06 · FND-07 Implemented

Upgrade complex pickers, ordering and tables

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	**T1 · 2.0 Alpha**	**Not yet delivered**	**Compose, Test Data, Processes & Packs, Control Object Repository, Execution Center**

ACCEPTANCE
3/3

Description

Implement accessible search, grouped selection, ordered transfer, drag alternatives and responsive tables.

User story

As a keyboard or assistive-technology user, I want complete complex-widget behavior.

Acceptance criteria

1	Grouped pickers implement an approved combobox or tree pattern.	Implemented
2	Every reorder action has keyboard controls and announcements.	Implemented
3	Tables preserve labels and actions at supported narrow widths.	Implemented

Implementation coverage

Grouped selection now uses a searchable combobox/listbox with Arrow, Home, End, Enter and Escape behavior. All draggable ordering surfaces also provide keyboard buttons and live announcements. Responsive tables expose row labels and retain actions.

Evidence: GroupedPicker.tsx · FileChainPicker.tsx · TestCaseEditor.tsx reorder alternatives · ObjectBrowser.tsx reorder alternatives · index.css responsive-table · t1-foundations.test.js keyboard journeys
Last reconciled: 2026-07-29

BL-017 · PB-031 · FND-09 · A11Y-01 · EXC-026 · EXC2-006 Implemented

Pass accessibility and production release verification

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T4 · 2.0 GA**	**Not yet delivered**	**Quality Engineering, All workspaces**

ACCEPTANCE
3/3

Description

Validate completed journeys to WCAG 2.2 AA and assemble production release evidence.

User story

As a product owner, I want objective accessibility and release evidence before approving 2.0.

Acceptance criteria

1	Axe has no unapproved serious or critical findings.	Implemented
2	Keyboard and NVDA journeys are recorded for every primary workspace.	Implemented
3	Responsive, security, secret-scan and approved live SAP evidence are signed.	Implemented

Implementation coverage

All seven primary workspaces pass Axe with no serious/critical findings, keyboard skip/focus entry, and 320 CSS-pixel reflow. A paced NVDA 2026.1.1 session in headed Chrome 150 recorded every workspace landmark, heading, control role, label and state against isolated data. A Chrome-only 6px Execution Center overflow was found, fixed and retested. The final GA gate passed build, typecheck, 57 core tests, 28 isolated API tests, 32 isolated UI tests and a 256-file pre-cleanup secret scan; the frozen 251-file release set was rescanned successfully. Technical accessibility, security and positive P2P/O2C evidence verification was signed by the verification executor and accepted by workspace owner Sathyanarayanan Kiran at 21:18:11 +05:30. BL-041's technical hold was resolved, and the workspace owner approved QA/4HANA Studio 2.0 for GA on 30 July 2026 at 08:00:18 +05:30.

Evidence: regression/ui/accessibility.test.js · regression/results/nvda/primary-workspaces-2026-07-29.log · NVDA_PRIMARY_WORKSPACE_RESULTS.md · ACCESSIBILITY_RELEASE_VERIFICATION.md · RELEASE_SIGN_OFF_2.0.md
Last reconciled: 2026-07-30

BL-018 · PB-017 · DASH-01 Implemented

Complete the task-focused Automation Overview

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	T4 · 2.0 GA	2.1.0	Automation Overview

ACCEPTANCE
3/3

Description

Turn Overview into a trusted landing page with direct continuation, alerts and scoped metrics.

User story

As a test manager, I want Overview to show recent work and issues needing attention.

Acceptance criteria

1	Selected tests and runs open their exact detail routes.	Implemented
2	Configured target status is accurate and actionable.	Implemented
3	Recent failures, stale drafts and blocked readiness are surfaced without duplicating navigation.	Implemented

Implementation coverage

A recent execution row now opens its exact audit record (/audit/runs/{id}) instead of the general Audit workspace, and the inspector's "Open in Compose" button opens the exact selected Test (/compose/tests/{file}) instead of the bare editor. A new Needs Attention panel surfaces exactly three real, non-invented signals without duplicating any existing browsing UI: executions that failed in the last 7 days, draft Tests with no steps yet, and Tests not yet published (and therefore blocked from Regression Packs) — each a single link into the workspace that already owns that data (Audit and Evidence / Test Library), reusing the existing library status computation rather than inventing a new one. Configured target status (already accurate/actionable via workspaceContext) required no change.

Evidence: packages/studio-web/src/components/AutomationOverview.tsx canvas-attention panel and onNavigateToRoute wiring · regression/ui/overview.test.js
Last reconciled: 2026-07-30

BL-019 · PB-034 · FUT-03 Implemented

Add filterable execution value and trend analytics

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	T4 · 2.0 GA	2.1.0	Automation Overview

ACCEPTANCE
3/3

Description

Scope outcomes, runtime, manual effort and TCO by time, target, process and test.

User story

As a test manager, I want transparent trends so that I can identify risk and realised automation value.

Acceptance criteria

1	Every metric states date range, filters, numerator, denominator and freshness.	Implemented
2	Cost assumptions are saved as owner workspace preferences.	Implemented
3	Charts have accessible tabular equivalents and never fabricate trends.	Implemented

Implementation coverage

The Execution impact dashboard now has real date-range and App ID filters (App ID options are derived from the recorded runs themselves, not the unrelated Object Repository app-id list), and a Scope line states the exact numerator (matching executions), denominator context (App ID/date range), analysis window size (up to 500 most recent executions) and freshness (as-of time). The outcomes metric now states its own numerator/denominator explicitly ("P of T passed (R%)") rather than only a percentage bar. Cost assumptions are no longer reset to hardcoded defaults on every reload: a new OverviewPreferencesStore (mirroring workspaceGovernance.ts's own JSON-file pattern) persists them as a shared owner workspace preference via GET/PUT /api/settings/overview-preferences, debounced client-side so typing doesn't spam the endpoint. A new Weekly trend section buckets the same filtered runs into real calendar weeks (Monday-start) and renders one accessible table (not a canvas chart) with a pass-rate bar per row — a week with no executions is simply absent, never interpolated, satisfying "never fabricate trends" by construction rather than by disclaimer.

Evidence: packages/studio-server/src/overviewPreferences.ts · packages/studio-server/src/server.ts overview-preferences routes · packages/studio-web/src/components/AutomationOverview.tsx computeWeeklyTrend/ExecutionImpactDashboard · regression/overview-preferences-store.test.js · regression/api/overview-preferences.test.js · regression/ui/overview.test.js
Last reconciled: 2026-07-30

BL-020 · PB-018 · TEST-01 Implemented

Provide a routeable Test Library

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	T2 · 2.0 Beta	2.1.0	Compose

ACCEPTANCE
3/3

Description

Find, create, duplicate, archive and open tests using business metadata rather than file-first controls.

User story

As a test author, I want a searchable Test Library so that I avoid duplicates and resume exact work.

Acceptance criteria

1	List supports name, process area, application and status filters.	Implemented
2	New Test captures business name, application, domain and blank/template choice.	Implemented
3	Opening or creating uses a stable test route while preserving JSON compatibility.	Implemented

Implementation coverage

Compose now opens a responsive Test Library with name/file search plus process-area, application and readiness filters. Guided creation captures the business name, generated or edited file name, application, process area and blank-versus-existing-Test starting point. Creation is server-persisted, conflict-safe and immediately opens a refresh-stable detail route; legacy Tests without application metadata remain valid and default to SAP in the library index.

Evidence: TestLibrary.tsx · TestCaseEditor.tsx application metadata · server.ts /api/testcases/library and POST /api/testcases/:file · api.ts listTestLibrary/createTestCase · testcases.test.js · test-library.test.js · compose.test.js · stable-routes.test.js · unsaved-guards.test.js
Last reconciled: 2026-07-30

BL-021 · PB-019 · TEST-02 · EXC-006 · EXC-007 · EXC2-003 · SC-01 · SC-02 · SC-03 Implemented

Build typed, validated Single Tests

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	T2 · 2.0 Beta	2.1.0	Compose

ACCEPTANCE
4/4

Description

Compose steps, objects, value sources, declared inputs/outputs and readiness in one guided canvas.

User story

As a functional consultant, I want guided test composition so that saved Tests are understandable and runnable.

Acceptance criteria

1	Steps distinguish literal, dataset, system-context and prior-output values.	Implemented
2	Tests visually declare typed inputs, outputs and sensitivity.	Implemented
3	Required objects, parameters, unresolved values and output collisions block publishing.	Implemented
4	Reordering is keyboard accessible and existing ModuleCall JSON round-trips.	Implemented

Implementation coverage

Compose now distinguishes literal, dataset, system-context and prior-output step values; visually authors typed Test inputs, outputs and sensitivity; and blocks publishing for unresolved required values, missing objects or parameters, and output collisions. Existing ModuleCall JSON remains executable and round-trips through the editor, while keyboard-accessible reordering is retained.

Evidence: TestCaseEditor.tsx · StepEditor.tsx · TestContractEditor.tsx · packages/core/src/domain/testContract.ts · server.ts Test publish validation · testcases.test.js · typed-test-authoring.test.js · compose.test.js
Last reconciled: 2026-07-30

BL-022 · PB-020 · OBJ-01 Implemented

Complete the Control Object Repository workbench

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	**T2 · 2.0 Beta**	**2.1.0**	**Control Object Repository**

ACCEPTANCE
4/4

Description

Search, inspect, edit, highlight and safely maintain reusable SAP UI5 objects.

User story

As an automation engineer, I want a central object workbench so that selectors remain reusable and understandable.

Acceptance criteria

1	Search spans domain, App ID, name, label, type and stability.	Implemented
2	Object detail shows selector metadata, scope, usage and last verification.	Implemented
3	Rename and delete show dependency impact and preserve or guide reference correction.	Implemented
4	Ordering has keyboard controls and does not require drag.	Implemented

Implementation coverage

Search now spans domain, App ID, name, label, type and stability (unstable/verification status/duplicate terms) in one box. Selecting an object shows a detail panel with its selector metadata (control id, binding path, table id), scope, created/updated/updated-by, last-verified state, and which Tests use it (fetched via the new dependency-index scan, precise per module's own objectKind schema — not a coincidental string match). Rename now propagates automatically into every referencing Test's step params (an IDE-style symbol rename, since it's the same control under a new name); delete blocks with the exact referencing Test list unless explicitly forced, since a removed reference can't be safely auto-repaired the way a rename can.

Evidence: ObjectBrowser.tsx detail panel and dependency-aware delete · packages/studio-server/src/server.ts findObjectUsage/renameObjectInTestCase · packages/core/src/domain/objectRepository.ts · regression/api/objects.test.js · regression/object-repository.test.js
Last reconciled: 2026-07-30

BL-023 · PB-021 · OBJ-01 Implemented

Complete guided live-screen capture

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	**T2 · 2.0 Beta**	**2.1.0**	**Control Object Repository, Compose**

ACCEPTANCE
4/4

Description

Verify the SAP target, capture or pick controls, curate names and save stable objects safely.

User story

As a functional consultant, I want guided capture so that objects are stable and stored under the correct application.

Acceptance criteria

1	Scan starts from the configured SAP URL and displays target/session health.	Implemented
2	Bulk capture and interactive picking identify exact duplicates before save.	Implemented
3	App ID is detected or explicitly confirmed for each capture.	Implemented
4	Contextual capture returns the saved object to the originating Compose field without losing work.	Implemented

Implementation coverage

Any object-kind Compose field now shows a persistent "+ Capture" action beside it (deliberately outside the suggestion dropdown, which is absolutely positioned and would otherwise cover it while open). It opens ContextualCapturePanel — an app-level sibling overlay, like SettingsPanel, not a route change — so it never touches the dirty-navigation guard or risks the in-progress, unsaved step edit underneath. The overlay prefills the field's App ID and shows which field it's capturing for; CurationList's "Use for <field>" action (on a just-saved or already-saved row) fills the field and closes the overlay, and the picker's own option list refetches immediately after. A pre-existing latent bug was fixed alongside this: the picker's suggestion dropdown never closed on blur, only on Escape/selection/click-outside — fixed by closing on blur with every actionable row's mousedown already preventing that blur from firing prematurely.

Evidence: ContextualCapturePanel.tsx · ObjectPicker.tsx +Capture action · CurationList.tsx captureTarget / Use for <field> · ObjectScanner.tsx captureTarget banner · StepEditor.tsx onRequestCapture wiring · App.tsx captureRequest overlay · regression/ui/compose.test.js contextual capture test
Last reconciled: 2026-07-30

BL-024 · New · Control Object Repository stocktake Implemented

Add selector health, history and governance

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	**T2 · 2.0 Beta**	**2.1.0**	**Control Object Repository, Audit & Evidence**

ACCEPTANCE
4/4

Description

Track selector quality, verification history, duplicates and change impact over time.

User story

As an automation maintainer, I want selector health and lineage so that object changes do not silently break Tests.

Acceptance criteria

1	Repository records created, updated and last-verified metadata.	Implemented
2	Health flags unstable IDs, missing live controls and likely duplicates.	Implemented
3	Reverification can compare live metadata without overwriting silently.	Implemented
4	Object changes are attributable and dependency-aware.	Implemented

Implementation coverage

Every control now records createdAt/updatedAt/updatedBy (OS username, same convention as RunHistoryStore's "who ran it") and a denormalized last-verified snapshot. A "Reverify" action opens the live scan session, matches the stored control by id (or, if drifted, by id-suffix + type — the same idea as FioriPlaywrightAdapter's healIdBySuffix), and records the outcome as a permanent history event without changing the stored control — an explicit "Update stored control to match" action is required to actually apply a drifted id, so nothing is overwritten silently. Health flags: unstable-id (mirrors ui5Inspector.ts's isAutoGeneratedId heuristic, duplicated rather than imported since core has zero dependency on adapter-fiori by design), missing (last reverify found nothing live), and likely-duplicate (same App ID + type + label under a different name). Rename/delete's dependency-awareness (shared with BL-022 AC3) satisfies "attributable and dependency-aware."

Evidence: packages/core/src/domain/objectRepository.ts isLikelyUnstableId/findLikelyDuplicates/recordVerification · packages/studio-server/src/scanSession.ts reverifyControl · ObjectBrowser.tsx health badges and verification history · regression/object-repository.test.js · regression/api/objects.test.js
Last reconciled: 2026-07-30

BL-025 · PB-022 · DATA-01 Implemented

Provide a routeable Test Data Library

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	**T2 · 2.0 Beta**	**2.1.0**	**Test Data**

ACCEPTANCE **4/4**

Description

Create, find and safely maintain flat datasets with schema and usage information.

User story

As a test-data author, I want a searchable data library so that reusable datasets are governed and understandable.

Acceptance criteria

1	Dataset list supports search, process area, format and stable routes.	Implemented
2	Columns have names, types, examples and sensitivity.	Implemented
3	Rename/removal shows affected Tests and Processes.	Implemented
4	Scalar, list and table cells survive RFC4180 round-trip.	Implemented

Implementation coverage

A new Dataset Library section (backed by GET /api/data/library) sits above Test Data's existing open/create controls, with search, process-area and format filters over every dataset's file, format, process area and row count. Columns declare a name/type/sensitivity/example via a new DataColumnSchemaStore (reusing the same TestValueType/DataSensitivity vocabulary as a Test's own contract inputs), edited inline per CSV column. Rename and removal reuse BL-022's dependency-aware pattern: findDataFileUsage scans Groups (Process dataFile), Regression Packs (member dataFile) and relational-CSV definitions for references — delete blocks with a 409 and the referencing list unless forced, rename propagates the new file name into every referencing artifact automatically (an IDE-style symbol rename) and both surface the affected Processes/Packs/relationships before the tester confirms. RFC4180 round-trip (AC4) was already covered by ListCell/TableRowsEditor and needed no further work.

Evidence: packages/core/src/domain/dataColumnSchema.ts · packages/studio-server/src/server.ts data library/usage/rename/schema routes · packages/studio-web/src/components/DataLibrary.tsx · packages/studio-web/src/components/DataEditor.tsx column schema section · regression/data-column-schema.test.js · regression/api/data-library.test.js · regression/ui/data-tab.test.js · regression/ui/stable-routes.test.js
Last reconciled: 2026-07-30

BL-026 · EXC-009 · EXC-010 · EXC2-002 · SC-05 · SC-06 · SC-17 · SC-18 Implemented

Author nested and relational transaction data

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T2 · 2.0 Beta**	**Not yet delivered**	**Test Data**

ACCEPTANCE
4/4

Description

Design one-header/many-item and many-header/many-item datasets without flattening transaction structure.

User story

As a test-data author, I want nested and relational models so that SAP document loops reflect real business data.

Acceptance criteria

1	UI authors nested JSON transaction arrays.	Implemented
2	UI relates header and child CSVs by declared keys and collection path.	Implemented
3	Duplicates, missing keys, orphans and collection collisions are actionable.	Implemented
4	Preview shows transaction and child counts before save.	Implemented

Implementation coverage

Test Data now authors nested JSON transaction arrays, persists header/child CSV relationship definitions, validates duplicate/missing/orphan/collision conditions through the authoritative loader, and shows transaction plus child counts before save. Joined samples remain nested and owned by their header transaction.

Evidence: packages/core/src/domain/dataSet.ts · packages/studio-server/src/server.ts data preview/relation APIs · packages/studio-web/src/components/DataEditor.tsx · regression/execution-data.test.js · regression/api/data.test.js · regression/ui/data-tab.test.js
Last reconciled: 2026-07-29

BL-027 · EXC-008 · EXC-011 · EXC-019 · EXC2-002 · SC-07 · SC-08 · SC-09 · SC-10 · SC-19 · SC-20 Implemented

Map, preview and snapshot effective execution data

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T2 · 2.0 Beta**	**Not yet delivered**	**Execution Center, Test Data**

ACCEPTANCE
4/4

Description

Bind reusable data to process, stage or Pack member scope and preview exact iterations.

User story

As an operator, I want to inspect effective mappings so that the execution consumes the intended values.

Acceptance criteria

1	Mapping supports process data, stage override, Pack-member data, literals, system context and prior outputs.	Implemented
2	Filters and maximum limits recalculate the execution matrix.	Implemented
3	Preview and Start use the same immutable snapshot.	Implemented
4	Invalid mappings block Start and link to the owning workspace.	Implemented

Implementation coverage

The versioned model supports process, stage, Pack-member, literal, system-context and prior-output mappings. Execution Center now authors typed nested-property filters and record limits, recalculates the authoritative matrix, displays every exact selected record plus its content hash and resolved input source, and seals that plan/data into the same immutable snapshot consumed by Start. Filter changes invalidate the preflight token; invalid mappings remain blocking with exact correction routes.

Evidence: packages/core/src/domain/executionPlan.ts applyDataSelection · packages/engine/src/executionOrchestrator.ts · packages/studio-server/src/executionPreflight.ts · packages/studio-web/src/components/RunPanel.tsx · regression/execution-plan-contract.test.js · regression/execution-preflight.test.js · regression/execution-orchestrator.test.js · regression/ui/run-tab.test.js
Last reconciled: 2026-07-29

BL-028 · PB-023 · COL-01 · EXC-001 · SC-04 · SC-11 · SC-12 · SC-13 · SC-14 · SC-15 · SC-16 Implemented

Separate Business Processes and Regression Packs

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	T3 · 2.0 RC	2.1.0	Processes & Packs

ACCEPTANCE
4/4

Description

Replace ambiguous Group/Suite concepts with two explicit reusable artifact types.

User story

As a test manager, I want dependency and isolation semantics in business language so that I choose the correct composition.

Acceptance criteria

1	Business Process means ordered dependent Tests with explicit hand-offs.	Implemented
2	Regression Pack means independent Tests or Processes with isolated contexts.	Implemented
3	Legacy Group, Chain, Suite and Batch inputs translate without behavior loss.	Implemented
4	The user-facing workspace is named Processes & Packs after approval.	Implemented

Implementation coverage

The user-facing Processes & Packs workspace now separates ordered Business Processes from persisted Regression Packs. Packs accept independent Test or Business Process members with isolated member data, application, session and failure-policy bindings. Existing Group, Chain, Suite and Batch translation remains backward-compatible.

Evidence: ProcessPacksWorkspace.tsx · PackEditor.tsx · GroupEditor.tsx · server.ts /api/packs · legacyExecutionPlans.ts · packs.test.js · packs-tab.test.js
Last reconciled: 2026-07-30

BL-029 · EXC-018 · EXC2-003 Implemented

Build Processes and Packs visually

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	**T3 · 2.0 RC**	**2.1.0**	**Processes & Packs, Compose**

ACCEPTANCE
4/4

Description

Compose sequence, independence, contracts, hand-offs, policies and member bindings without editing plan JSON.

User story

As a process author, I want a visual canvas so that execution topology and data movement are obvious.

Acceptance criteria

1	Process canvas shows stages and typed output-to-input hand-offs.	Implemented
2	Pack canvas shows independent member lanes and member-specific bindings.	Implemented
3	Validation identifies unresolved inputs, cycles, duplicate outputs and incompatible policies.	Implemented
4	Keyboard composition, version status and publishing are supported.	Implemented

Implementation coverage

The visual Business Process canvas now persists ordered stages, typed Test contracts and explicit process-data, literal, system-context or prior-stage output bindings. Validation rejects unresolved required inputs, forward or cyclic references, unknown and duplicate outputs, type mismatches and incompatible Pack session policies. Keyboard-native Process and Pack composition, versioned draft/published lifecycles, stable routes and direct mixed Saved Pack execution are covered by isolated API and browser journeys.

Evidence: GroupEditor.tsx visual topology and typed hand-offs · PackEditor.tsx · ProcessPacksWorkspace.tsx · server.ts /api/groups and /api/packs validation · executionPreflight.ts saved Pack planning · packages/cli/src/commands/batch.ts · groups.test.js · packs.test.js · runs.test.js · groups-tab.test.js · packs-tab.test.js · run-tab.test.js
Last reconciled: 2026-07-30

BL-030 · PB-024 · EXE-01 · EXC-017 · EXC-020 · EXC-021 Implemented

Complete authoritative preflight and execution review

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T2 · 2.0 Beta**	**Not yet delivered**	**Execution Center**

ACCEPTANCE
4/4

Description

Validate target, artifacts, data, policies and calculated work through one server-owned model.

User story

As an operator, I want authoritative preflight so that configuration errors are corrected before SAP opens.

Acceptance criteria

1	Server validates target, credentials, versions, contracts, objects, mappings and policies.	Implemented
2	Review shows members, iterations, stages, steps and known child work.	Implemented
3	Blocking findings link to the owning artifact; warnings require acknowledgement.	Implemented
4	Start token and plan hash prevent stale or duplicate starts.	Implemented

Implementation coverage

Authoritative server preflight validates target freshness, credential availability, plan/contract versions, repository objects, data mappings and execution policies. Review presents the full calculated matrix and child count; correction routes and warning acknowledgement are enforced; token, plan hash, target hash and duplicate-claim handling protect Start.

Evidence: executionPreflight.ts · RunPanel.tsx · regression/execution-preflight.test.js · regression/ui/run-tab.test.js
Last reconciled: 2026-07-29

BL-031 · PB-025 · RUN-01 · EXC-002 · EXC-003 · EXC-013 · EXC-022 · EXC-027 · EXC2-004 · SC-22

Implemented

Complete hierarchical execution monitoring

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	T3 · 2.0 RC	2.1.0	Execution Center

ACCEPTANCE
4/4

Description

Monitor Pack, Process, Test, Iteration, Step and Child Item from persisted events.

User story

As an operator, I want the complete hierarchy so that progress and failures never appear flattened or stalled.

Acceptance criteria

1	All six hierarchy levels have stable identity, status, totals and duration.	Implemented
2	Known child work contributes measurable progress.	Implemented
3	Refresh/reconnect rebuilds the same state from persisted events.	Implemented
4	Monitor, history, metrics and evidence counts reconcile.	Implemented

Implementation coverage

Every level now has a stable, persisted identity: Pack (executionId) and Process (memberId) already had one; Test/Stage now carries the plan's own stageId (RunStage.stageId), Step is stable within its stage (StepResult.stepId, e.g. "step-2"), and Child Item's last-known state (label/completed/total/currentKey/status) now survives on the completed StepResult itself (StepResult.childWork) instead of only living in a transient, overwritten progress.json. A real correctness bug was found and fixed for reconciliation: a mid-run iteration that fails before its Test even starts writes no run-N.json and no evidence-manifest entry, and reading results back by array position (instead of by each file's own iteration number) silently attributed a LATER iteration's result and evidence to an EARLIER iteration's row. buildHierarchy now keys results by their own "run-N.json" number and ranks evidence documents the same way, so Monitor's per-iteration display is correct even across a gap — proven by a regression test that deliberately skips a middle iteration and asserts no cross-attribution.

Evidence: packages/core/src/domain/runResult.ts stageId/stepId/childWork · packages/engine/src/executionEngine.ts, executionOrchestrator.ts · packages/studio-server/src/runs.ts readIndexedResults/buildHierarchy · regression/execution-hierarchy-diagnosis.test.js
Last reconciled: 2026-07-30

BL-032 · PB-026 · FAIL-01 · EXC-023 Implemented

Provide focused failure diagnosis

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	**T4 · 2.0 GA**	**2.1.0**	**Execution Center, Audit & Evidence**

ACCEPTANCE
4/4

Description

Explain the first actionable failure with safe context and links to correction.

User story

As an investigator, I want focused diagnosis so that I can correct the cause without reading raw logs.

Acceptance criteria

1	Failure location identifies Process, Test, iteration, step and child item.	Implemented
2	Expected/actual context is redacted according to policy.	Implemented
3	Links open the exact Test, object, dataset, mapping or setting.	Implemented
4	Raw diagnostics remain secondary and safe.	Implemented

Implementation coverage

FailureDiagnosis now carries a correction (kind/route/label), computed from the plan's own snapshot rather than guessed: an object-repository failure's control name is recovered from the message's own double-quoted convention (every such throw site already quotes the name — never invented) and links straight to that control in the Object Repository (/objects/{appId}/{name}); a data-binding failure links to the exact dataset file; every other category (setup, authentication, navigation, assertion, execution) links to the originating Test. Monitor renders this as a labelled button next to the diagnosis message, reusing the same onNavigateToRoute plumbing already proven by the preflight correction links. Verified end-to-end: a forced synthetic failure in the isolated harness renders "Root failure · object", the exact message, and a working "Open … in the Control Object Repository" link that navigates to the precise route.

Evidence: packages/studio-server/src/runs.ts correctionFor/findFailedTestAsset/findMemberDataFile · packages/studio-web/src/components/RunPanel.tsx correction button · regression/execution-hierarchy-diagnosis.test.js · regression/api/runs.test.js forced-failure diagnosis · regression/ui/run-tab.test.js forced-failure correction link
Last reconciled: 2026-07-30

BL-033 · PB-027 · RERUN-01 · EXC-016 · EXC2-005 · SC-21 Implemented

Rerun safely with lineage and explicit differences

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T4 · 2.0 GA**	**Not yet delivered**	**Execution Center, Audit & Evidence**

ACCEPTANCE
4/4

Description

Review failed/full rerun scope and compare it with the immutable source execution.

User story

As an operator, I want traceable reruns so that recovery does not duplicate business side effects.

Acceptance criteria

1	Rerun review compares plan, data, policies, target and scope.	Implemented
2	Failed-only rerun is offered only where runtime semantics permit.	Implemented
3	New run records parent, reason, scope and changed inputs.	Implemented
4	Cancellation, retry and resume do not duplicate completed side effects.	Implemented

Implementation coverage

Rerun is now a two-step review-and-confirm workflow that compares immutable plan, selected data, session/failure policies, current verified target and requested scope. The review hash prevents post-review drift; the new run persists parent, reason, scope, reviewed hash and every changed input. Failed-only scope is disabled when no eligible work exists, excludes passed records even inside a failed Pack member, and blocks any started transactional iteration whose SAP state must be retained. Full transactional replay is likewise blocked when it could duplicate documents. Cancellation remains at the active-transaction boundary and request keys make retry idempotent.

Evidence: packages/core/src/domain/executionPlan.ts transaction policy snapshot · packages/studio-server/src/runs.ts reviewRerun and createRerunSnapshot · packages/studio-server/src/server.ts rerun-review endpoint · packages/studio-web/src/components/RunPanel.tsx rerun difference review · regression/execution-recovery.test.js · regression/api/runs.test.js · regression/ui/run-tab.test.js
Last reconciled: 2026-07-29

BL-034 · PB-028 · EXC-024 · EXC2-001 · SC-26 Implemented

Secure canonical evidence and provenance

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T1 · 2.0 Alpha**	**Not yet delivered**	**Audit & Evidence, Execution Center**

ACCEPTANCE
4/4

Description

Protect one immutable evidence source and expose safe provenance from both run and audit views.

User story

As an auditor, I want secure canonical evidence so that records are trustworthy and not duplicated.

Acceptance criteria

1	Execution Center and Audit resolve the same archived evidence artifact.	Implemented
2	Artifact access requires authenticated owner authorization.	Implemented
3	Evidence shows run, target class, time zone, plan/data versions and redaction state.	Implemented
4	Missing evidence explains its lifecycle reason without leaking existence.	Implemented

Implementation coverage

Execution Center and Audit reuse one owner-protected archive artifact. New PDFs record run/execution identity, target class and verification time, time zone, plan/snapshot/data versions and enforced redaction state; the Audit library presents retention/redaction policy and a safe lifecycle explanation when no PDF exists.

Evidence: evidencePdf.ts provenance metadata · server.ts protected artifact mounts · /api/evidence-governance · DocumentsPanel.tsx · evidence-reference.test.js · t1-security-context.test.js
Last reconciled: 2026-07-29

BL-035 · PB-029 · HIST-01 · EXC-024 Implemented

Complete searchable Audit and Evidence history

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P1	**T4 · 2.0 GA**	**2.1.0**	**Audit & Evidence**

ACCEPTANCE
4/4

Description

Find immutable runs, inspect stable details and traverse source/rerun lineage.

User story

As a test manager or auditor, I want meaningful history search so that I can answer delivery and compliance questions.

Acceptance criteria

1	Server supports date, outcome, artifact, App ID, environment, run ID and executor filters.	Implemented
2	Results paginate and sort by start, duration and status.	Implemented
3	Every result opens a stable detail route with parent/child lineage.	Implemented
4	Evidence types and source artifacts are linked from the run record.	Implemented

Implementation coverage

The audit ledger (RunHistoryStore) now supports every named filter server-side — date range, outcome, mode, artifact (Test/Group name), App ID, environment (target hostname/safety class), run ID and executor — plus a combined free-text search kept alongside them for the existing search box. list() returns a page plus a total count (surfaced via an X-Total-Count header so the existing bare-array response shape stays compatible with other consumers), with sort by started/duration/status in either direction. Lineage is now real: the CLI threads the Studio execution id (studioRunId) and, for a rerun, the source execution's id (parentStudioRunId) into every ledger row it writes, so the audit record can group an execution's own sibling iterations and, for a rerun, link straight back to the execution it was rerun from — both exposed in the detail panel as "View this execution's other runs" / "View source execution". Evidence types and source artifacts are now linked from the run record too: testCaseFiles (added alongside the existing display-name-only testCaseNames) let each chip open its Test directly, and a new GET /api/audit/runs/:id/documents surfaces the run's own captured business-document values (DocumentLog, now filterable by runId) alongside its canonical PDF.

Evidence: packages/core/src/domain/runHistoryStore.ts · packages/core/src/domain/documentLog.ts · packages/cli/src/commands/run.ts, suite.ts, batch.ts · packages/studio-server/src/server.ts audit routes · packages/studio-web/src/components/DocumentsPanel.tsx · regression/run-history-store.test.js · regression/api/audit-runs.test.js · regression/ui/audit-library.test.js
Last reconciled: 2026-07-30

BL-036 · PB-030 · SET-01 · EXC-025 · EXC2-001 Implemented

Secure identity, target and integration administration

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T1 · 2.0 Alpha**	**Not yet delivered**	**Settings, Shell, Execution Center, Control Object Repository**

ACCEPTANCE
4/4

Description

Protect the sole-owner workspace and server-side SAP integration without exposing credentials.

User story

As the workspace owner, I want secure integration settings so that execution and scanning use known targets safely.

Acceptance criteria

1	Authenticated owner identity protects mutations, runs and artifacts.	Implemented
2	Credentials remain server-side and are never returned, logged or committed.	Implemented
3	SAP target has URL, profile reference, safety class and verification status.	Implemented
4	Existing workspace data remains visible when the owner links a Google account.	Implemented

Implementation coverage

Single-owner authentication protects APIs and artifacts; Google linking retains the existing workspace. Credentials stay in the server-side vault/encrypted fallback and are redacted from logs. Settings now persists explicit target classification and performs a non-transactional login-only verification; executions and reruns capture the initiating owner and verified target snapshot.

Evidence: auth.ts · workspaceGovernance.ts · sapVerification.ts · SettingsPanel.tsx · runs.ts · CLI --executed-by and target provenance · sap-settings.test.js · t1-security-context.test.js
Last reconciled: 2026-07-29

BL-037 · PB-033 · FUT-02 · New cross-workspace requirement Implemented

Add global artifact search and dependency impact

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P2	**T5 · Post-2.0**	**2.1.0**	**Automation Overview, Compose, Test Data, Processes & Packs, Control Object Repository, Audit & Evidence**

ACCEPTANCE
4/4

Description

Search Tests, objects, datasets, Processes, Packs and runs and understand their relationships.

User story

As a maintainer, I want global search and dependency information so that changes are safe and duplicates are avoided.

Acceptance criteria

1	Search returns typed results with domain, application and lifecycle status.	Implemented
2	Detail views show verified incoming and outgoing usage.	Implemented
3	Rename, delete and archive use dependency information.	Implemented
4	Search respects owner and future role authorization.	Implemented

Implementation coverage

GET /api/search scans Tests, the Control Object Repository, Test Data, Processes, Regression Packs and the audit run ledger from a single query, returning typed results with domain (process area tag), application (App ID) and lifecycle status, plus the exact stable route to open each one. A header-level Global search overlay (reachable from every workspace, matching the SettingsPanel/ContextualCapturePanel sibling-overlay pattern) offers debounced search-as-you-type with kind badges and per-result meta. Each result exposes an incoming/outgoing usage panel: Tests show which Processes/Packs reference them and which Objects they reference (findTestUsage/findTestReferences); Processes show which Packs reference them (findGroupUsage) and their own Test/Dataset members; Objects and Datasets reuse BL-022/BL-025's existing usage scanners. Rename and delete are dependency-aware for every mutable kind: a blocked delete returns 409 with the exact referencing artifacts unless force=true, and a rename propagates the new file name into every referencing Process/Pack (the same IDE-style symbol rename established for Objects and Datasets) rather than leaving a dangling reference. Regression Packs have no possible dependents, so their delete/rename routes exist for a consistent search experience but need no usage scan. This product has no separate archived lifecycle for these artifact kinds (only draft/published),

so the AC3 wording's "archive" has no corresponding action to implement; rename and delete cover every dependency-aware mutation this workspace actually offers. The whole endpoint sits behind the same blanket owner-authentication gate as the rest of the API, verified by an anonymous-request test.

Evidence: packages/studio-server/src/server.ts findTestUsage/findGroupUsage/findTestReferences and GET /api/search · packages/studio-web/src/components/GlobalSearchPanel.tsx · packages/studio-web/src/App.tsx header search trigger and sibling overlay · regression/api/search.test.js · regression/api/testcases.test.js dependency-aware rename/delete · regression/api/groups.test.js dependency-aware rename/delete · regression/api/packs.test.js delete/rename · regression/ui/global-search.test.js · regression/t1-security-context.test.js anonymous /api/search check
Last reconciled: 2026-07-30

BL-038 · FND-08 · EXC-004 · EXC-005 · EXC-012 · EXC-014 · EXC-015 · EXC-028 · EXC2-006 · SC-23 · SC-24 · SC-25

Implemented

Protect compatibility, migration and execution correctness

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T4 · 2.0 GA**	**Not yet delivered**	**Quality Engineering, Execution Center**

ACCEPTANCE
4/4

Description

Retain legacy behavior through one versioned orchestration model and comprehensive regression coverage.

User story

As a product owner, I want migration and correctness tests so that redesign does not change intended execution behavior.

Acceptance criteria

1	Legacy Chain, Suite and Batch translate to versioned plans.	Implemented
2	Every selected Batch row executes and transaction outputs remain isolated.	Implemented
3	Policies, Pack scheduling, cancellation and evidence counts have regression coverage.	Implemented
4	Build, typecheck, isolated UI, secret scan and approved live cases pass before GA.	Implemented

Implementation coverage

Versioned translation, unified orchestration, every-row Batch behavior, isolation, policies, cancellation and evidence counts have regression coverage. The repository-wide migration audit translates every persisted Test and Group without source mutation, validates the combined Batch Pack, and preserves Suite order and independent identity. Build/typecheck, 57 core tests, 28 isolated API tests, 32 isolated UI tests and the high-confidence secret scan are green. Read-only live modes, controlled headed P2P/O2C and the authorised negative remediation are recorded. Run 299669f8-75b6-471d-9c56-ed0375011fc5 passed with immutable retained-state evidence, completing the approved live cases and criterion 4.

Evidence: executionPlan.ts · legacyExecutionPlans.ts · regression/legacy-migration.test.js · regression/secret-scan.mjs · package.json test:release:isolated · regression/results/quality-history.json · audit-evidence/299669f8-75b6-471d-9c56-ed0375011fc5/evidence.pdf · QA/4HANA Test Operations catalogue
Last reconciled: 2026-07-30

BL-039 · PB-032 · FUT-01 Deferred

Add multi-user roles and capabilities

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P3	**T5 · Post-2.0**	**Not yet delivered**	**Platform, All workspaces**

ACCEPTANCE
Deferred

Description

Introduce server-enforced permissions only when multi-user deployment is approved.

User story

As a product owner, I want role-based capabilities so that users see and perform authorised actions.

Acceptance criteria

1	Implementation starts after a confirmed multi-user requirement.	Deferred
2	Server protects every mutation, execution, artifact and administration action.	Deferred
3	Role mapping covers manager, consultant, engineer, tester, analyst and auditor.	Deferred

Implementation coverage

The product intentionally uses one preserved owner identity for 2.0, and this remains true for 2.1: formally excluded from 2.1.0 scope by the 31 Jul 2026 release-qualification decision (see RELEASE_GOVERNANCE_2.1.md §9) — no multi-user requirement has been confirmed, so this item is not part of the 2.1 candidate and does not block its GA readiness.

Evidence: Single-owner product decision · RELEASE_GOVERNANCE_2.1.md §9 (2.1.0 scope exclusion)
Last reconciled: 2026-07-31

BL-040 · FUT-04 Deferred

Add scheduling, notification and controlled parallelism

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P3	**T5 · Post-2.0**	**Not yet delivered**	**Execution Center, Automation Overview**

ACCEPTANCE
Deferred

Description

Schedule approved work, notify outcomes and parallelise only where isolation is proven.

User story

As a test manager, I want scheduled and parallel runs so that regression throughput scales safely.

Acceptance criteria

1	Schedules capture immutable scope, target and owner.	Deferred
2	Notifications link to stable run records without exposing secrets.	Deferred
3	Parallel work respects target capacity, data isolation and cancellation.	Deferred
4	Concurrency limits and queue health are observable.	Deferred

Implementation coverage

Deliberately outside the 2.0 release; Pack member scheduling semantics exist only for current sequential orchestration. Formally excluded from 2.1.0 scope by the 31 Jul 2026 release-qualification decision (see RELEASE_GOVERNANCE_2.1.md §9) — isolation for parallelism has not been proven, so this item is not part of the 2.1 candidate and does not block its GA readiness.

Evidence: Approved 2.0 scope decision · RELEASE_GOVERNANCE_2.1.md §9 (2.1.0 scope exclusion)
Last reconciled: 2026-07-31

BL-041 · T1.3.1 · Execution validation closure Implemented

Close authoritative Chain, Suite and Batch validation

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	**T1 · 2.0 Alpha**	**Not yet delivered**	**Quality Engineering, Execution Center**

ACCEPTANCE
4/4

Description

Replace assumed or skipped execution coverage with recorded isolated and non-production SAP results while separating read-only validation from business-document transactions.

User story

As the product owner, I want trustworthy mode-level test evidence so that T1.4 starts only after the execution heart of the product is proven.

Acceptance criteria

1	The Test Operations catalogue is generated from recorded test-run output and never fabricates status, time or failure history.	Implemented
2	Isolated API and Studio UI journeys complete Chain, Suite and multi-group Batch with synthetic execution and no SAP side effects.	Implemented
3	Read-only API and Studio UI journeys complete Chain, Suite and two-group Batch against a live-verified non-production SAP target.	Implemented
4	Transactional P2P and negative cases have approved test-data ownership, unique run references, fail-stop behavior, retained-state evidence and a recorded pass. Any remediation is a separate explicitly authorized activity.	Implemented

Implementation coverage

The 125-test Test Operations catalogue is generated from repository inventory plus recorded Node output; executable reconciliation proves that status, timestamp, duration and retained failure history are not fabricated. Core (57), isolated API (28), isolated UI (32), live read-only API (3 modes) and live read-only Studio UI (3 modes) are green. Controlled headed P2P and O2C runs passed with owner-linked references, fail-stop policy and canonical evidence. The first controlled negative attempt, eb060bc6-673b-4e65-8919-6ccf66b6a645, remains immutably failed and retained. After explicit owner authorisation, remediation run 299669f8-75b6-471d-9c56-ed0375011fc5 passed all six steps under reference Q4HP2P2607309366: the title remained New Purchase Order, SAP displayed "Document contains no items," no document number or downstream step occurred, and state remained retain-for-review. The recorded pass is ingested into Test Operations. All four criteria are complete.

Evidence: regression/reporters/result-capture.js · regression/record-quality-run.mjs · regression/quality-catalog.test.js · regression/results/quality-history.json · regression/results/raw/live-negative-remediation-2026-07-30.ndjson · regression/transactional-negative.test.js · packages/engine/src/modules/assertDocumentCreationBlocked.ts · testcases/create-po-missing-item-negative.json · regression/api/runs.test.js · regression/ui/run-tab.test.js · audit-evidence/d55cd0f0-07ed-433f-8e40-1b61cd102827/evidence.pdf · audit-evidence/4b4f15d7-32df-4348-be70-

23fe5243ff7a/evidence.pdf · audit-evidence/eb060bc6-673b-4e65-8919-6ccf66b6a645/evidence.pdf · audit-evidence/299669f8-75b6-471d-9c56-ed0375011fc5/evidence.pdf · QA/4HANA Test Operations catalogue
Last reconciled: 2026-07-30

BL-042 · HC-013 · Post-production hypercare Implemented

Add sortable fields to Compose

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P3	**T5 · Post-2.0**	**2.1.0**	**Compose**

ACCEPTANCE
3/3

Description

Let a tester reorder Compose's own field list (not just step order) by dragging or keyboard, for forms with many parameters.

User story

As an automation engineer authoring a Test with many fields, I want to sort or reorder them so that I can organize the form the way the target screen presents it.

Acceptance criteria

1	A Compose field list supports reordering independent of the underlying JSON key order.	Implemented
2	Reordering has keyboard controls and does not require drag (consistent with BL-016's ordered-transfer pattern).	Implemented
3	Reordering persists across save/reload.	Implemented

Implementation coverage

Raised as a post-production hypercare New Feature request (HC-013). A step's own parameter list is now reorderable from the keyboard: each free-form parameter carries Move up / Move down controls, matching the ArrowUp/ArrowDown idiom TestCaseEditor already uses for step order, with an aria-live announcement so a reorder is not visible-only. Persistence needed no schema change and no separate ordering field — JSON.stringify emits string keys in insertion order, so rebuilding the params object in the chosen order is itself what makes the order survive save and reload. Deliberately scoped to modules with no descriptor: a described module's fields render in the order its own describe.params declares, which is part of that module's contract and identical in every Test that uses it, so it is not a per-step preference to override. The free-form keys are different — they exist only in this Test's JSON, which is exactly why their order is this step's to choose. Regression test drives the reorder through the UI and then re-reads the stored Test, so the assertion is on what persisted rather than on what the open editor displayed.

Evidence: Post Production Incidents.csv (HC-013) · packages/studio-web/src/components/StepEditor.tsx · regression/ui/backlog-t5-usability.test.js
Last reconciled: 2026-08-10

BL-043 · HC-017 · Post-production hypercare Implemented

Add multi-select to Mass Capture before saving

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P3	**T5 · Post-2.0**	**2.1.0**	**Control Object Repository**

ACCEPTANCE
3/3

Description

Let a tester select exactly which discovered controls to save from a Mass Capture result set, instead of saving everything not individually dismissed.

User story

As an automation engineer capturing a busy screen, I want to pick just the controls I need so that I don't clutter the Object Repository with ones I'll never use.

Acceptance criteria

1	Each Mass Capture result row has an explicit selection control.	Implemented
2	"Save selected" saves only checked rows; existing "Save all" and per-row Cancel remain available.	Implemented
3	Selection state survives a Highlight-on-screen round trip.	Implemented

Implementation coverage

Raised as a post-production hypercare Defect (HC-017). Every savable Mass Capture row now has its own checkbox, with "Save selected (n)" and a Select all / Clear selection toggle alongside the existing controls. "Save all" and per-row Cancel are untouched on purpose rather than reimplemented in terms of selection: they are a different intent, and a bulk capture where you genuinely want everything should not first require ticking forty boxes. The third criterion drove the implementation more than the first two — selection is keyed by controlId in a Set, not by row index or DOM order, because Highlight-on-screen re-renders the list while the user is mid-selection and dismissing a row shifts every index after it; anything positional would have silently reset the selection and saved the wrong set. Rows that cannot be saved (no stable id, or already saved) render an empty cell rather than a dead checkbox, so the control is only ever offered where it does something. The component is shared with the picker flow, which is why the change is additive throughout and no existing path changed behaviour.

Evidence: Post Production Incidents.csv (HC-017) · packages/studio-web/src/components/CurationList.tsx · regression/ui/backlog-t5-usability.test.js
Last reconciled: 2026-08-10

BL-044 · HC-021 · Post-production hypercare Implemented

Turn the Process area field into a suggest-dropdown

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P3	T5 · Post-2.0	2.1.0	Compose, Processes & Packs, Test Data, Control Object Repository

ACCEPTANCE
3/3

Description

Replace the free-text Process area field's plain input with a combobox that shows the current value selected among known process areas.

User story

As a tester tagging a Test or Process, I want to see and pick from existing process areas in a dropdown so that I don't retype or slightly misspell an existing one.

Acceptance criteria

1	The Process area control shows known values as selectable options with the current value indicated.	Implemented
2	Typing a new value remains possible (the set of process areas is not fixed).	Implemented
3	Applied consistently everywhere the field appears (Compose, Business Processes, Test Data, Control Object Repository).	Implemented

Implementation coverage

Raised as a post-production hypercare Defect (HC-021, "Open Text > Domain"). The Process area control is now a real combobox — an input backed by a datalist, deliberately not a select. That distinction is the requirement, not an implementation detail: the set of process areas is a growing convention rather than a fixed enumeration, so the control has to offer everything that already exists AND still accept something new in the same keystroke; a select would have made the first use of a new area impossible. The current value is indicated rather than merely present, via aria-pressed on the matching quick-pick chip. The chips stay alongside the dropdown because they save in one click where choosing from the list still requires pressing Save. Consistency across Compose, Business Processes, Test Data and the Control Object Repository came free: all four render the same DomainTag component, so the change landed in one place instead of four. Each field generates its own datalist id — several tagged artifacts can share a screen, and a shared id would let one field's suggestions silently drive another's. Superseded on 13 Aug 2026 in the G-Stride line: the control is now a dropdown of process area folders plus a "+ New process area" entry, and Save appears only once the selection differs from what is stored. The original rationale for insisting on a combobox was that "a <select> would make the first use of a new area impossible" — that objection is answered by the create entry, which registers the folder and files the artifact into it without leaving the screen, so the criteria above still hold: known values are offered, a brand-new one is still reachable, and the current value is indicated by

being the selected option rather than by a pressed chip. The test was rewritten to assert the new contract, including that a new area persists to both the tag store and the folder registry. This is a divergence from the criteria as originally worded and is flagged for owner ratification rather than treated as settled.

Evidence: Post Production Incidents.csv (HC-021) · packages/studio-web/src/components/DomainTag.tsx · regression/ui/backlog-t5-usability.test.js
Last reconciled: 2026-08-13

BL-045 · HC-024 · Post-production hypercare Not started

Improve Test Data grid row readability

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P3	T5 · Post-2.0	Not yet delivered	Test Data

ACCEPTANCE
0/3

Description

Reduce visual clutter in the dataset row-editing grid so cell values are clearly readable at a glance.

User story

As a tester editing a wide dataset, I want the row grid to be easy to scan so that I can verify values quickly.

Acceptance criteria

1	Column values are legible without expanding every cell.	Open
2	Dense per-cell controls (edit/expand icons) remain available but visually secondary.	Open
3	Change does not regress the existing nested-JSON and relational CSV editing flows.	Open

Implementation coverage

HELD by owner decision on 10 Aug 2026 — "let us circle back to this". Not a scheduling deferral: the item needs a design decision first (what to trade away — rows per screen, inline editability, or column count), and building to a guess would be worse than waiting. Raised as a post-production hypercare Defect (HC-024). Investigated: the current grid is a dense, always-editable table by design, needed for bulk data entry across typed/relational datasets — a real readability trade-off, not a code bug. Scoped here for a considered visual redesign rather than an unreviewed change to a shared editing surface.

Evidence: Post Production Incidents.csv (HC-024) · packages/studio-web/src/components/DataEditor.tsx
Last reconciled: 2026-07-31

BL-046 · HC-033 · Post-production hypercare Not started

Group and organize Audit and Evidence at scale

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P3	T5 · Post-2.0	Not yet delivered	Audit & Evidence

ACCEPTANCE
0/3

Description

Give the Audit and Evidence run list a grouping or virtualization strategy so it stays usable as recorded runs accumulate into the thousands.

User story

As an auditor with a long-running workspace, I want runs grouped meaningfully so that I don't have to scroll through everything to find what I need.

Acceptance criteria

1	Long result sets are grouped (e.g. by date or App ID) or virtualized instead of one continuous scroll.	Open
2	Existing filters, sort, pagination and lineage links keep working unchanged.	Open
3	Behavior is verified against a realistically large synthetic run set, not just a handful of fixtures.	Open

Implementation coverage

Raised as a post-production hypercare Defect (HC-033). BL-035 already added pagination and every AC1 filter, which helps, but no grouping/virtualization strategy exists yet for very large ledgers. ATTEMPTED AND REVERTED on 10 Aug 2026. The owner supplied reference designs (a navigation tree with roll-up counts per node, a filtered grid, a details panel), and an implementation was built to them: a Process area → App ID tree keyed off the existing appId tag store, with pass rate and passed/failed/total per node computed across the whole filtered ledger by a new aggregate query, node selection behaving as a filter so every existing filter/sort/pagination/lineage kept working inside it, and an explicit bucket for untagged App IDs. It passed its own regression tests and, against the real 114-run ledger, did surface something the flat list hid (createOutboundDelivery failing 20 of 23). The owner reviewed it in the running product and judged the module worse off than before, and it was reverted in full at their instruction — server aggregate, API route, client types, tree UI, styles and tests. The revert was surgical: HC-029's evidence-PDF change and BL-050's harness fix shipped in the same commit and were kept. What is NOT yet known, and should be established before any second attempt, is which part made it worse — the tree competing with the existing filter toolbar for the same job, the vertical space it took above the run list, the Process area → App ID hierarchy being the wrong axis, or the roll-ups themselves. Building again without that answer would risk repeating the same outcome, so this stays Not started rather than being re-scoped on a guess.

Evidence: Post Production Incidents.csv (HC-033) · Owner reference designs, 10 Aug 2026 · Reverted implementation: f77e08c, ae81507 · packages/studio-web/src/components/DocumentsPanel.tsx

Last reconciled: 2026-08-10

BL-050 · Found while closing the 2.1.0 live-SAP gate, 10 Aug 2026 Implemented

The documented way to run the live-SAP gate cannot actually reach the tenant

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P2	T5 · Post-2.0	2.1.0	Platform, Execution Center

ACCEPTANCE
3/3

Description

RELEASE_VERIFICATION_2.1_CHECKLIST §3 tells the release owner to close the live-SAP gate with `REGRESSION_LIVE=1 npm run test:api:isolated`, but the isolated harnesses hardcode a synthetic SAP target and never read REGRESSION_LIVE — so that command cannot exercise the real tenant.

User story

As the release owner, I want the documented verification command to actually perform the verification it claims, so that a gate cannot be recorded as closed on evidence that never touched the target.

Acceptance criteria

1	The documented command for the live-SAP gate runs against the real configured target, or the checklist documents the command that does.	Implemented
2	Running the live suites cannot silently pass against a synthetic host — an unreachable or mis-targeted live run fails loudly.	Implemented
3	The synthetic-fixture tests in the same files do not fail merely because the live mode was requested.	Implemented

Implementation coverage

Found on 10 Aug 2026 while actually closing the §3 gate for the 2.1.0 GA rather than by reading the checklist. Both regression/run-isolated-api.js and regression/run-isolated-ui.js set the SAP target to https://synthetic.non-production.invalid against their own throwaway credential store, and neither file references REGRESSION_LIVE anywhere — so the checklist's documented command exercises a fake host no matter what the environment variable says. This is a governance defect rather than a test defect: the risk is not that the command errors, it is that a gate whose whole purpose is proving the product works against real SAP could be recorded as closed on a run that never reached it. The gate was genuinely closed for 2.1.0 by running the live cases directly against a server using the owner's real credential store (REGRESSION_LIVE=1 REGRESSION_ALLOW_EXECUTION=1 node --test on the two files), and all six read-only live cases passed — see RELEASE_SIGN_OFF_2.1.md. The checklist has been annotated with the working command in the meantime, but the harnesses themselves should honour the flag so the documented and the effective procedure stop diverging. A secondary annoyance to fix with it: run in that mode, the synthetic fixtures in those same files fail, because they only exist inside the isolated harness — which makes a correct live run look like a failing one to anyone reading the raw output. Fixed on 10 Aug 2026, and the defect ran deeper than the original diagnosis: the harness was not only pointing at a synthetic host, it was also

substituting a synthetic run service, a verifySap stub that unconditionally returned verified, a throwaway credential store, a forced file-only credential fallback (the real credentials resolve from the OS store), an isolated governance record holding the target's safety classification, and synthetic Test fixtures in place of the real Test files the live suites actually drive. Every one of those had to be conditional for the documented command to mean anything — so live mode now stops isolating the workspace altogether and runs the ordinary server with execution enabled, which is what the checklist always meant. Two guards keep it honest rather than merely working: the run refuses to start unless a target is genuinely configured, and refuses outright if that target is not classified non-production. In live mode only the live-gated cases run (--test-name-pattern=^live), because the synthetic-fixture tests in those same files have no fixtures against a real workspace and their failures previously buried the three results that matter. Verified end to end: REGRESSION_LIVE=1 npm run test:api:isolated now prints the real target it selected and exits 0 with all three read-only live cases passing against the tenant. The --test-name-pattern restriction is a safety property as much as a signal-to-noise one, and that was learned the hard way during this fix: an intermediate build ran the FULL suite against the real workspace, and the artifact-lifecycle tests duly created, renamed and deleted real Tests, Processes and datasets in it — leaving 19 stray files and one Process pointing at a Test that no longer existed, which then failed legacy-migration.test.js. Nothing committed was touched and the strays were removed with git clean, but the episode is the reason live mode must never run anything except the live-gated cases: once the harness stops isolating the workspace, every non-live test that writes is writing to the owner's real one.

Evidence: docs/ui-ux/RELEASE_VERIFICATION_2.1_CHECKLIST.md §3 · regression/run-isolated-api.js · regression/run-isolated-ui.js · docs/ui-ux/RELEASE_SIGN_OFF_2.1.md
Last reconciled: 2026-08-10

BL-049 · Found during BL-047 validation, 4-10 Aug 2026 Implemented

Release workflow validated a catalogue a later suite regenerated

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P2	**T5 · Post-2.0**	**2.1.0**	**Platform, Test Operations**

ACCEPTANCE
3/3

Description

The recorded core suite asserted that the committed test catalogue matched the test sources, but the catalogue was only regenerated after that suite had already run — so any change to the set of tests failed the first run and passed the second.

User story

As the release owner, I want a failing release check to mean something is actually wrong, so that I read failures instead of learning to re-run past them.

Acceptance criteria

1	The catalogue the core suite validates is current for the run in progress, not left over from the previous one.	Implemented
2	A commit that adds, renames or removes a test passes the recorded workflow on its first run.	Implemented
3	The check still fails for genuine catalogue drift rather than being weakened or removed.	Implemented

Implementation coverage

Found while running the release workflow repeatedly during BL-047 and its removal, and initially mis-attributed — including by me — as a known flaky test. It is not flaky, it is deterministic: quality-catalog.test.js asserts that the committed test-catalog.json matches a fresh scan of the test sources, but run-recorded-core.js regenerated that catalogue only AFTER the suite finished, so the suite was always checking the previous run's file. Any commit that changed the set of test names therefore failed on the first run and passed on the second, having regenerated the file in between. The cost was not the extra minutes: it produced false failures in release evidence often enough to train a re-run reflex, which is exactly how a real catalogue drift would have been waved through. Fixed by generating the catalogue before the suite as well as after. Running it twice is correct rather than redundant — the first pass fixes the inventory half, which depends only on the test sources and is what a test-name change breaks; the pass after the run fixes the status half, which depends on results that do not exist yet beforehand. The check itself is unchanged and still fails on genuine drift. Confirmed in practice on 10 Aug 2026: the first workflow run after the test set stopped changing passed the catalogue check on its first attempt, and the runs in this pass — which added and renamed tests — did the same.

Evidence: regression/run-recorded-core.js · regression/quality-catalog.test.js
Last reconciled: 2026-08-10

BL-048 · New requirement — de-brand the product Implemented

Remove the aielk logo and "by aielk" attribution from the product and its artefacts

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P2	T5 · Post-2.0	2.1.0	Shell, All workspaces, Audit & Evidence

ACCEPTANCE
5/5

Description

Strip the aielk deer logo and every "by aielk" / "AI ELK" attribution out of the product: the app shell and login screen, the browser favicon and page titles, generated evidence PDFs and product guides, and the repository's own metadata and documentation. What replaces the brand in each location is an open decision (see criteria) — the item is not just a delete.

User story

As the product owner, I want no aielk branding anywhere in the product or anything it generates, so that the platform and its evidence carry no third-party attribution.

Acceptance criteria

1	The logo is gone from the app shell, the login screen and the browser favicon/apple-touch icon, with a decided replacement (plain wordmark, product mark, or nothing) rather than an empty gap where it used to sit.	Implemented
2	No "by aielk" or "AI ELK" string remains in the running product, in generated evidence PDFs, in the generated product guides, or in repository metadata (package.json description, page titles, CSS comments).	Implemented
3	The five live logo image assets are removed from the repository, and no code path still references a deleted file. The two under audit-evidence/ are screenshots of historical design evidence, not live assets, and are retained under criterion 5.	Implemented
4	Regression coverage is updated rather than dropped: regression/ui/overview.test.js currently asserts the logo IS visible in the persistent shell, so it must be changed to assert the new intended state.	Implemented
5	Already-signed 2.0.0 evidence under audit-evidence/ is left untouched, per RELEASE_GOVERNANCE_2.1 §'v2.0.0 remains frozen and unchanged' — historical evidence records what was true when it was recorded.	Implemented

Implementation coverage

Raised by the workspace owner on 10 Aug 2026. Footprint established by search, not estimated: 7 logo image assets (packages/studio-web/public/ai-elk-logo-{transparent,dark,evidence}.png, apps/test-operations/public/ai-

elk-logo-{transparent,dark}.png, audit-evidence/logo-{light-transparent,dark-contrast}.png) and roughly 30 source references across 9 files. The UI touchpoints are App.tsx's brand-logo-pair and brand-wordmark, LoginScreen.tsx's login-logo-pair, index.html's three icon links plus meta description and title, and about fifteen CSS rules in index.css. Generated artefacts matter as much as screens: evidencePdf.ts embeds the logo as a data URI and prints "QA/4HANA STUDIO · by aielk" on every evidence PDF footer, and generate-product-pdfs.mjs puts it on each product-guide cover and running header — so the three committed guide HTMLs regenerate clean once the generator is fixed. Repository metadata (package.json description) and doc prose (CANVAS_FIRST_IMPLEMENTATION.md, this tracker) also carry the name. Two things make this more than a find-and-replace. First, removal leaves real holes — the shell header, the login screen and the PDF cover are laid out around a logo, so what goes in its place is a design decision, not a deletion. Second, audit-evidence/ holds already-signed 2.0.0 evidence PDFs with the logo baked in; RELEASE_GOVERNANCE_2.1 forbids modifying frozen 2.0.0 evidence, so those stay as they are and the item is scoped to the product and everything it generates from here on. Implemented on 10 Aug 2026 to the owner's instruction that nothing replaces the logo and the product simply reads "QA/4HANA Studio". The shell header and login screen now carry the wordmark alone; the collapsed sidebar shows "QA/4" where the mark used to sit, so collapsing still leaves something legible rather than an empty rail. Favicon and apple-touch links are gone rather than repointed, since there is no replacement mark to point at. Evidence PDFs and the three product guides regenerate clean — the guides were rebuilt as part of this change, not left to drift. Twenty dead CSS rules for the removed logo and wordmark elements were deleted with them. Five live image assets removed; the two under audit-evidence/ are July screenshots of historical design evidence, not live assets, and are retained deliberately — deleting them would falsify a record of what the product looked like at the time. The one remaining occurrence of the name anywhere in the product's own documentation is this item's own text, which necessarily quotes what it removed. regression/ui/overview.test.js's branding assertion was inverted rather than deleted: it now requires the wordmark and asserts no image is present in the shell brand area, so a logo reappearing fails instead of passing unnoticed. Corrected the same day after the owner pointed out that this tracker's OWN page header still carried the logo and a "by aielk" byline — it had been marked 5/5 while a visible reference remained. The cause was a bad search on my part, not a judgement call: the sweep used a regex requiring 60 characters of context on both sides of the match, which silently skips any occurrence near the start of a line, and the header markup sat at indentation. Re-swept without that constraint; the header img (pointing at an already-deleted asset, so rendering broken) and the byline are gone, along with the now-dead .brand img CSS rule. Every remaining occurrence of the name in the repository is inside BL-048's own record and the change-log entry describing the removal — text that cannot describe what it removed without naming it.

Evidence: Workspace owner requirement, 10 Aug 2026 · packages/studio-web/src/App.tsx · packages/studio-web/src/components/LoginScreen.tsx · packages/studio-web/index.html · packages/studio-web/src/index.css · packages/reporting/src/evidencePdf.ts · scripts/generate-product-pdfs.mjs · apps/test-operations/app/layout.tsx · package.json · regression/ui/overview.test.js
Last reconciled: 2026-08-10

BL-047 · New requirement — autonomous AI-driven test authoring Deferred

Natural-language-driven test authoring: reuse what's known, autonomously discover what isn't

PRIORITY	PLANNED TRANCHE	ACTUAL RELEASE	WORKSPACE
P0	T5 · Post-2.0	Not yet delivered	Control Object Repository, Compose, Test Data, Execution Center

ACCEPTANCE
0/4

Description

A natural-language process name (e.g. "Create Purchase Requisition") resolves to one of two paths: if the App ID and Objects already exist, route straight into them exactly as today's manual workflow does; if the process is unknown to the platform, the engine autonomously discovers it live against the configured SAP tenant, captures its controls into the Object Repository, composes an executable Test, and generates and attaches Test Data — with a human reviewing and approving the result before it's treated as done. Additive to, not a replacement for, the existing manual capture/compose/curate workflow.

User story

As an automation engineer, I want to name a process in plain language and have Studio either open what it already knows or learn a new one directly from the live SAP tenant, so that I never have to manually scan screens, hand-author steps or hand-build test data the first time a process is automated — while still reviewing and approving anything the engine discovers on its own.

Acceptance criteria

1	A natural-language process name routes to an existing App ID/Test when real, captured Objects already exist for it — no discovery attempted, no behavior change from today's manual workflow.	Deferred
2	When the App ID/process is unknown, the engine launches the configured SAP tenant, locates a previously-created business document of the matching type as a live reference, and extracts reusable master data from it, without a human selecting either.	Deferred
3	The engine drives the live process end-to-end using that master data, capturing every control it actually interacts with into the Object Repository through the same upsert path (real createdAt/updatedAt, real duplicate/unstable-id detection) a manual capture uses — never a fabricated or scripted control definition.	Deferred

4	A Test is composed from the exact discovered interaction sequence and a Test Data instance is generated from the extracted master data and attached to it, both presented in a review screen for the owner to accept, edit or reject before either is treated as final; the same design generalises to other processes (e.g. Create Production Order) without process-specific code.	Deferred

Implementation coverage

Raised and prioritised directly by the workspace owner on 31 Jul 2026, after identifying that the existing createOutboundDelivery/Post Goods Issue Objects (and, on inspection, every one of the 53 controls in the real Object Repository — login, Create Purchase Order, Create Sales Order, Create Outbound Delivery, Post Goods Receipt, Post Supplier Invoice, Create Billing Document) has a NULL created_at/updated_at, which ObjectRepository.upsert() can never produce — meaning none of it was ever captured through the product's own capture path, and RELEASE_SIGN_OFF_2.0.md's O2C live-run evidence rests on Objects that were never actually verified by this product's own definition of verified. The owner reviewed the design and resolved its four open decisions (full detail: docs/ui-ux/AUTONOMOUS_TEST_AUTHORING_DESIGN.md), then asked to start implementation. Phase 1 (routing + reconciliation) is now built and regression-tested, though it does not yet satisfy any of the four acceptance criteria above — those describe the Autonomous Discovery Engine itself (Phase 2+), and acceptanceProgress correctly stays 0/4 until that exists. What Phase 1 delivers: (a) the Process Intent Router, extending Global Search (BL-037) — a query matching no existing Test/Object/Dataset/Process/Pack/Run now shows a suggested camelCase App ID and a direct route into Compose to start authoring manually, rather than a plain "no results"; (b) POST /api/objects/:appId/reconcile and a new "Reconcile all" action in the Control Object Repository — runs the existing reverifyControl() (the same check Reverify already does for one Object) across every stored Object for an App ID against whatever the open scan session currently has live, recording real verification history for each, so a screen Studio already knows doesn't collect a duplicate capture. Both reuse existing infrastructure exactly as the design specifies — no new SAP access, no new capture mechanism, no shortcut on provenance. Phase 2 has started: QueryValidLineItemData (the one real, working reference-document-lookup pattern found dormant in the engine, hardcoded to Purchase Orders and never regression-tested) is now generalized into QueryReferenceDocument — the same OData V2 header(+optional item) lookup, parameterized by service path/entity/filter/key/field-map so it works for any business object with an "active entity" OData service, not just Purchase Orders, with 6 new regression tests covering header-only and header+item lookups, missing-document/missing-item/malformed-fieldMap/missing-field error paths, and item-over-header field precedence — coverage QueryValidLineItemData itself never had. Phase 2's second piece is now built: per the owner's explicit choice ("rules first, model as fallback"), packages/engine/src/discoveryNavigation.ts decides the single next action on an unfamiliar Fiori screen using a small, fixed, auditable decision tree per screen archetype instead of a model call — classifyScreenArchetype() reads the Fiori elements template generator's namespace embedded directly in every control id on the screen (a real signal, verified from createOutboundDelivery's own captured controls) to tell List Report, Object Page and dialog screens apart; decideNextAction() then dispatches to a per-archetype rule modelled directly on the one piece of real ground truth available — the hand-written create-delivery.json flow's own search-then-select-then-act shape and createOutboundDelivery's real control ids — never guessing when a rule can't find what it needs, returning an explicit needsFallback result instead. A genuine logic bug surfaced by the new tests before this could be trusted: the List Report rule's "click the primary action" step could select the search screen's own Go button a second time (since only the higher-level "search" history entry, not a per-control click entry, had been recorded for it), which would have re-run the search instead of advancing the process; fixed by excluding the Go button from primary-action candidates. No model/LLM call is wired up anywhere in this module — deliberately deferred as its own decision (provider, credentials, cost) rather than assumed. Phase 2's third piece closes the loop from a decision to a real, live action: packages/studio-server/src/discoverySession.ts is the single-step live orchestrator the owner's "human in the loop" decision calls for — one capture-decide-register-execute cycle per request, never an unattended multi-step loop. Each cycle captures the live scan session's current screen,

classifies and decides via discoveryNavigation.ts, resolves the raw controlId the decision names to a real Object Repository logical name (reusing one already registered for that exact controlId, or deriving a new one via the new packages/engine/src/discoveryRegistration.ts naming heuristic and upserting it through the SAME path a human capture uses — real createdAt/updatedAt, real duplicate/unstable-id detection, never a fabricated definition, closing the exact trust gap this whole backlog item exists to fix), then executes the resolved ModuleCall against the live page via a new FioriPlaywrightAdapter.attach(page) static factory (drives the SAME browser window the scan session already has open, instead of launching a second, disconnected one). Two more real defects surfaced by building this, both fixed before anything could rely on them: (1) SelectTableRow requires a captured table *column* (with tableId set), never the table control itself — discoveryNavigation.ts's List Report rule was targeting the raw table control, which would have thrown "is not a table column" on the very first live attempt; fixed by adding CapturedControl.tableId and matching on a captured column instead. (2) The orchestrator needs to know what "having done this" should be called in history terms (search vs select-row vs fill:X vs click:X) without re-deriving each rule's own logic — NavigationDecision's action variant now carries an explicit historyKey the rule itself chooses, rather than the orchestrator guessing. A minimal UI panel ("Autonomous discovery" in Control Object Repository, gated on an open scan session) lets the owner actually drive this: enter reference values as key=value pairs, Start, then Run next step repeatedly, seeing each decision, registered control and outcome before continuing. 12 new regression tests (7 for deriveControlName's naming/collision rules, 5 for the new /api/discovery/* routes' guard behavior) plus 1 new UI test confirming the panel only appears once a scan session is open. Separately, while validating this pass, found and fixed a genuine (unrelated) race in regression/ui/data-tab.test.js's own HC-023 test: its baseline wait matched on any \d+ of \d+ datasets text, which the Dataset Library legitimately renders as "0 of 0" for one frame before its async fetch resolves — under load the test could read that transient state back with .textContent() before the real one landed; hardened to wait specifically for a non-zero denominator. Still not started: wiring the model fallback, the multi-step loop actually running end-to-end against a real live SAP tenant, Test composition and Test Data attachment from the discovered steps, and the mandatory review-and-approve gate UI — all remain open, and acceptanceProgress correctly stays 0/4 until at least one criterion is proven true end-to-end, not just built and unit/API-tested. First live test against a real tenant (createPurchaseRequisition, 3 Aug 2026) surfaced the real size of the remaining gap, which the owner then explicitly clarified is not process-specific scope creep to fix one-by-one but a general-solution requirement: given ANY plain-English requirement for ANY SAP process, the platform must resolve it, not just execute already-known screen navigation. Concretely still missing: (a) shell/Launchpad screen handling — classifyScreenArchetype only recognises List Report/Object Page/dialog, so a scan session opened straight to the Fiori Launchpad home screen correctly reports needsFallback and stops rather than guessing, but nothing can navigate off it yet; (b) the natural-language entry point itself — the built discovery panel takes manually-typed key=value reference values, not English text; (c) QueryReferenceDocument is real and tested but wired to nothing in this flow. Per the owner's clarification, (a)-(c) cannot be solved by more fixed rules for the general case — matching arbitrary English phrasing to an app and reasoning over a tenant's own OData service catalog needs real language understanding, unlike on-screen navigation's fixed Fiori-namespace signal. Reviewed model cost (Anthropic Haiku 4.5/GPT-5.4 mini/Gemini 3.1 Flash-Lite, all sub-$0.002/call at this task's token volume) and performance (Claude Opus 5/GPT-5.6/Gemini 3 Pro flagship tier) with the owner; decided on Claude Haiku 4.5 for the POC on cost. Built the model-integration foundation this pass, deliberately scoped to plumbing only, not yet wired into any discovery decision: packages/core/src/aiCredentials.ts stores a provider API key with the identical security properties SAP credentials already have (OS-native credential store first, AES-256-GCM encrypted file fallback, environment-variable override for CI) but as its own parallel module rather than a generalisation of credentials.ts, so this newer, less-hardened code can never affect the already-proven SAP credential path. packages/engine/src/aiResolver.ts defines a one-method AiResolver interface (provider-agnostic by design, since every vendor's "cheap tier" model has already been swapped at least once this year per the owner's own cost research); packages/studio-server/src/anthropicResolver.ts is the one concrete implementation, calling Anthropic's Messages API with Haiku 4.5 by default, its fetch function injectable so tests never need a real key or network access. New Settings section ("AI provider (BL-047 POC)") and /api/settings/ai-provider GET/PUT/DELETE routes let the owner paste a real key

in, encrypted the same way, never returned to the browser. 17 new regression tests: 4 for aiCredentials' encrypted round-trip/env-precedence/removal (spawned-process tests, mirroring credentials-fallback.test.js exactly), 4 for AnthropicResolver's request shape and error handling (mocked fetch), 4 for the new Settings routes, 1 UI test round-tripping the Configured badge, plus 4 already-existing tests unaffected. A secret-scan false positive surfaced and was fixed along the way: two test fixtures using an sk-ant-shaped literal (deliberately realistic) tripped the "literal credential assignment" rule; renamed to include "synthetic" per the scanner's existing allowlist convention, same one credentials-fallback.test.js already uses. Recorded core (118), isolated API (84, 4 live-gated) and isolated UI (59, 4 live-gated) all pass. The owner then verified the foundation live: after entering a real Anthropic API key in Settings, a direct AnthropicResolver.complete() call against the real (not mocked) API succeeded, confirmed before building anything on top of it. First concrete consumer of the model now built: packages/engine/src/discoveryNavigation.ts gains a 'shell' ScreenArchetype for the Fiori Launchpad home screen, detected by the presence of sap.m.GenericTile/sap.f.Card controls — an already-verified signal in this codebase (ui5Inspector.ts's ACTIONABLE_TYPES/AUTO_ID_EXEMPT_TYPES/BORROWS_CHILD_TEXT entries for these exact two types were each added after a real capture on this tenant's own Launchpad, not guessed up front). decideShellAction() is the one archetype whose decision calls the model rather than a fixed rule, precisely because matching arbitrary tile labels to an arbitrary process has no fixed structural signal the way List Report/Object Page do: it builds a numbered list of every tile's visible text, asks the model (via the now-pluggable AiResolver) to reply with only the matching tile's number or NONE, and treats anything else — an unparseable reply, an out-of-range number, NONE, or a tile already clicked once without the screen advancing — as an explicit needsFallback rather than a guess. humanizeAppId() reverses BL-037's existing camelCase-App-ID convention ('createPurchaseRequisition' -> 'Create Purchase Requisition') to give the model a readable phrase to match against, since every discovery run already has an App ID by construction even before the natural-language entry point exists. decideNextAction() is now async and takes an optional AiResolver, threaded through from discoverySession.ts's runDiscoveryStep(), which only constructs a real AnthropicResolver when a key is actually configured (checked via getAiCredentialStatus each step) — with none configured, decideShellAction's own check gives a clear, actionable needsFallback message instead of a crash. 8 new regression tests for the shell archetype and decideShellAction (model-match success, NONE reply, unparseable reply, out-of-range number, already-clicked tile, no tiles with visible text) plus the pre-existing 15 List Report/Object Page/Dialog tests updated to await the now-async decideNextAction. Still not started: the natural-language entry point itself (still manual key=value reference values today), QueryReferenceDocument wired to anything in this flow, proving the Launchpad-to-app hop against a real live tenant end-to-end, Test composition/Test Data attachment, and the mandatory review gate — acceptanceProgress correctly stays 0/4 until at least one criterion is proven true live. First real live attempt against the owner's tenant (Launchpad -> createPurchaseRequisition) produced a genuine, useful finding rather than a clean pass: the loop correctly clicked a 'ProcurementControl' category tile on the first step, but then stopped on the resulting screen with 'no tile confidently matched "Create Purchase Requisition"' — the owner shared a screenshot of the actual live browser window showing the real tile was there all along, labelled "Process Purchase Requisitions", among others ("Manage Purchase Orders", "Post Goods Receipt for Purchasing Document", "Monitor Purchase Requisition Items", ...). The model had been too literal: it wouldn't match "Create" to "Process" even though both name the same business object. Fixed by extending buildTileSelectionPrompt() with explicit guidance that SAP Fiori tiles are conventionally named after the business object plus a generic verb (Process/Manage/Post/Monitor/Track) rather than the exact action requested, while also cautioning the model not to over-correct into picking a tile for a different lifecycle stage of the same object (the prompt names "Monitor Purchase Requisition Items" as an explicit example of a wrong pick to avoid). 2 new regression tests lock in this exact real scenario (the six real tile labels from the owner's screenshot) — one asserting the prompt actually carries the new guidance, one asserting the model's pick of "Process Purchase Requisitions" resolves to the correct action. Recorded core (129), isolated API (84, 4 live-gated) and isolated UI (59, 4 live-gated) all pass. A second live retry immediately surfaced a related but distinct finding: this time the loop stopped on the very first step, on a different screen — the top-level "My Home" Launchpad landing page, whose tiles are broad department/category cards ("Finance", "Manufacturing and Supply Chain", "Procurement", "Project Management", "Sales", "Other"), not

specific app tiles at all. None of these literally or even loosely names "Create Purchase Requisition", so the model correctly found no direct match — but it had no way to know that picking "Procurement" to drill into first (exactly what worked in the earlier successful run) was itself the correct move, since the prompt only ever described tiles as if they were all specific apps. Fixed by extending the prompt further: it now explicitly describes this two-level Launchpad structure (department/category tiles vs specific app tiles) and instructs the model to pick the most relevant department tile when no specific app tile is visible, naming "Procurement" as the example for anything about purchase requisitions/orders/goods receipt/supplier invoices. 2 more regression tests lock in the exact six "My Home" tile labels from the owner's second screenshot — one confirming the prompt carries the new category guidance, one confirming the model's pick of "Procurement" resolves correctly. Recorded core (131), isolated API (84, 4 live-gated) and isolated UI (59, 4 live-gated) all pass. A third retry immediately surfaced a related but distinct finding: the loop stopped on its own duplicate-click safety check ("already clicked the matching tile ('Procurement') once without navigating away"), even though the click had genuinely worked — a screenshot pair showed the Procurement tab now active and its own app tiles visible, including "Process Purchase Requisitions". The model had re-picked "Procurement" again instead of the now-visible, more specific tile, because a persistent quick-nav "Procurement" tile and the tab's own app tiles were both present in the same capture, and the prompt had no priority order between a category tile and a specific tile when both appear together — it only ever said "pick a category tile when no specific one exists", which technically wasn't the case here. Fixed by adding an explicit priority rule: a specific app tile always outranks a category tile whenever both are visible, with "Procurement" vs "Process Purchase Requisitions" as the concrete named example. 2 more regression tests lock this in with a fixture combining both tile types in one capture, confirming the specific tile wins. Recorded core (133), isolated API (84, 4 live-gated) and isolated UI (59, 4 live-gated) all pass. Three real live findings in a row, each fixed same-session on real evidence rather than guessed at — this prompt-based tile matching is proving to need real-world iteration the same way any new heuristic does, exactly why it stayed scoped to the Launchpad-navigation step and not assumed to generalise untested. A fourth retry got two real steps deeper than any before it — Procurement, then Process Purchase Requisitions itself, landing on the real List Report screen with its own filter fields and results table — before stopping on a genuinely different class of bug, not another prompt-wording gap: 'already clicked the matching tile ("Process Purchase Requisitions") once without navigating away', even though the screenshot clearly showed a real, different screen now on-screen. Pulled the actual live capture directly via /api/scan/capture against the still-open scan session to diagnose rather than guessing, and found the real cause: the Fiori Launchpad keeps a previously-opened app's whole UI5 component alive in the DOM (hidden, not destroyed) after navigating away — a standard SAPUI5 "keep alive" performance pattern, not a tenant defect. The live capture, taken while sitting on the Process Purchase Requisitions screen, still contained 40 sap.m.GenericTile controls left over from the Home page (bound under "homeApp-component---myhome--..."), all still resolving through core.byId() — including "Procurement" and "Process Purchase Requisitions" itself, which is exactly what the model kept re-matching against instead of the real screen's own content, since only 1 control in the entire 1260-control capture even contained "ListReport" in its id (this app is not built on the classic Fiori Elements V2 template discoveryNavigation.ts's list-report/object-page detection targets, so it correctly fell through to "no fixed-rule archetype matched" — but the stale tiles wrongly won the shell check instead). This is a bug in the shared capture pipeline itself, not in discoveryNavigation.ts's decision logic: packages/adapter-fiori/src/ui5Inspector.ts's inspectUi5Controls() now filters to only controls whose DOM element is actually rendered (not display:none, not inside a hidden ancestor), excluding stale kept-alive components regardless of which app they belonged to or what they're named. Since this module is shared by manual capture and Reconcile as well as discovery, every one of those gets the same correctness improvement, not just this feature. Added the first-ever regression test for this previously-untested module (regression/ui5-inspector-visibility.test.js) — a synthetic UI5-core stub page via Playwright (no real Fiori app needed) proving a visible control is captured and both a self-hidden and an ancestor-hidden control are excluded, the same shape the real kept-alive bug took. Recorded core (134), isolated API (84, 4 live-gated) and isolated UI (59, 4 live-gated) all pass. After the visibility fix, the loop reached the real Process Purchase Requisitions screen and correctly reported "Screen did not match any known Fiori elements archetype" — genuine progress (the app is built on a template discoveryNavigation.ts's list-report/object-page rules don't

recognise, not a bug), but the owner asked directly whether this stop-and-wait pattern was actually working and whether the model should be extended to handle it, rather than staying scoped to Launchpad tiles alone. Confirmed the design was working as intended (four straight live findings, each a genuine same-session fix on real evidence, and the system had never once acted with unwarranted confidence) and recommended extending the model to this case too, since it's the same architecture, not a new one; the owner agreed. Built decideUnknownAction() as BL-047's second model-driven archetype: fillableFields()/clickableButtons()/selectableTables() extract candidates from the capture by control-type pattern (the same fillable-input/button/table-column distinctions the fixed-rule archetypes already use, just without a fixed id pattern to key off), the model is shown each list plus the process's reference data and must reply in exactly one of four fixed shapes — "FILL <field> <reference key>", "CLICK <button>", "SELECT <table>", or "NONE" — with any other reply, an out-of-range index, an unknown reference key, or a target already acted on this screen all resolving to an explicit needsFallback rather than a guess, the same discipline decideShellAction already established. 14 new regression tests use field/button/column labels ("Purchase Requisition Number", "Plant", "Material Group", "Go", "Create Purchase Order", "Purchase Requisition"/"Material ID" columns) taken directly from the real Process Purchase Requisitions screen reached live, covering every one of the four reply shapes, all the reject-and-fall-back paths, and confirming no model call happens at all when a screen has nothing fillable/clickable/selectable to act on in the first place. Recorded core (145), isolated API (84, 4 live-gated) and isolated UI (59, 4 live-gated) all pass. The next live retry then exposed the design's real limit, and the owner called it correctly: with decideUnknownAction() live, the loop stopped stopping — but what it did instead was worse, taking seven consecutive ClickButton steps (ProcurementControl, ProcessPurchaseRequisitionsControl, InternalBtnButton, TriggerButton, SaveAsButton, SaveButton, and SaveButton a second time) with no coherent business flow between them, adding whatever the screen happened to offer rather than following any process. This was not another prompt-wording bug and could not be fixed by another prompt tweak: on every call the model was handed only an abstract key=value processContext dictionary of reference values, with no natural-language statement of what the run was actually trying to accomplish and no memory whatsoever of the steps it had already taken — so it could not know the goal, could not tell mid-flow from finished, and could not recognise it had already saved. The owner's direction was to make the model the platform's foundation for adding objects and composing tests, driven by a plain-English instruction rather than the key-value model, with an explicit two-step priority: (1) the model independently traverses the screen and registers the relevant controls into the Object Repository, and only once that is proven working, (2) a shell test case is generated in Compose from those objects. Step 2 is deliberately not started. Step 1 is now rebuilt around that: processContext is gone entirely, replaced by an instruction string (the owner's own example: "Goto project management, click on manage my timesheet and select the task. Enter 5 hours for today and save & submit") plus a stepLog the orchestrator appends a human-readable narration to after every successful action, both threaded into every model call so it always sees the whole goal and everything already done toward it. The four previously separate archetype decision paths (decideListReportAction, decideObjectPageAction, decideShellAction, decideUnknownAction) collapse into one decideWithModel() used for every non-dialog archetype, with the archetype now only contributing a one-line hint to the prompt rather than selecting a different code path — dialogs remain the single fixed rule, since clicking the one actionable button needs no instruction interpretation. The deterministic List Report/Object Page rules were deleted rather than kept: without processContext they had no data source left, and in every live run this whole engagement they had never once fired, because every real screen reached was either the Launchpad or an unrecognised archetype — a disclosed simplification, not an oversight. The model now replies in one of five shapes — FILL <n> <literal value>, CLICK <n>, SELECT <n>, DONE, or NONE — with FILL taking a literal value the model extracts from the instruction itself ("5" from "enter 5 hours") rather than a lookup key into a dictionary that no longer exists, and DONE as a new terminal state letting the model declare the instruction fully carried out instead of only ever being stopped by a safety check. The three hard-won lessons from the live Launchpad rounds (generic SAP verb naming, category tiles being valid picks, specific tiles outranking category tiles) were folded into the single unified prompt rather than lost with the code that used to hold them. The API surface changed to match end to end: POST /api/discovery/:appId/start now requires a non-empty instruction string instead of a processContext object, and

the Autonomous discovery panel's key=value box is replaced by a single plain-English instruction textarea carrying the owner's example as its placeholder. Model selection moved twice in the same pass on the owner's direction: Haiku 4.5 (the original POC choice, made purely on cost) to Sonnet 5 once the task became multi-step instruction following with its own memory, then to Opus 5 once the owner judged that reliability on exactly this kind of agentic multi-step decision-making mattered more than the remaining fraction of a cent — still trivial in absolute terms at roughly $0.02 a call, well under $1 for a full 20-step process run. discovery-navigation.test.js was rewritten for the new signature (29 tests) including one that exists solely to prove the step log motivating this entire rewrite actually reaches the prompt. The first live run of the instruction-driven loop (My Timesheet, 4 Aug 2026) took two steps and stalled with 'Already clicked "undefined"', and the owner raised three things: it did not progress much; a human takeover should have its steps recorded; and "Why do we still have run next step? If I am going to sit there and click run next step, I might as well add the controls myself, where is the AI here?" Pulling the live capture directly rather than guessing gave a conclusive answer to the first, and it was not a model-quality problem at all — it was a candidate-surface one. The screen's four tasks ("Administration Tasks", "Miscellaneous", "Training", "Travel Times") were captured perfectly, with stable ids and correct titles, as sap.m.ObjectListItem — a type not in ACTIONABLE_TYPES, so they classified as merely informational and were never offered as something that could be acted on. The instruction said "select the task" and the one thing it needed was invisible. Worse, 8 of the screen's 11 buttons had no text at all (icon-only: alertBtn, copyBtn, settingsBtn, groupTasks, msgPopoverBtn), and the prompt builder's `text ?? controlId` fallback was showing the model raw control ids to choose between; it picked one, which registered into the Object Repository as "Button" and reported back as the literal string "undefined". So the model's entire menu on that screen was a date-range button, Save & Submit, Close My Tasks, eight unreadable ids and one search field — no correct move existed. Three fixes, all in the shared capture pipeline or the candidate rules rather than the prompt: the whole sap.m list-item family joins ACTIONABLE_TYPES (which subtype a Fiori list uses is a rendering choice, not a semantic one); ui5Inspector gains a tooltip-then-icon-name label fallback for icon-only control types, applied strictly after classifyControl and with the raw fields stripped before anything downstream sees them, so a tooltip can improve what a control is called without ever changing what it is (the same discipline enrichWithChildText already followed, and the reason ALWAYS_STRUCTURAL_TYPES exists); and an unlabelled control is now simply not a candidate at all, because a raw id is not something a model can choose between on merit and not something worth a name in the repository — with every needsFallback reason going through the same labelOf() helper so "undefined" cannot recur. On the second and third points the owner was right on both counts. The single-step gate was the correct call while one decision at a time still needed watching, but as the product it is just a slower manual capture, so runDiscovery() now drives the instruction to completion on its own: it runs detached from the request that started it (a real run far outlasts any HTTP timeout), the UI polls state, and the human-in-the-loop guarantee moves to where it belongs — a Stop that takes effect between steps and never mid-action, a step budget so a confused run cannot spin against a live tenant indefinitely, and the mandatory review gate before anything is final. Stop now also means two different things on purpose: mid-run it halts the loop but keeps everything found, and only once nothing is running does it discard the run. Single-stepping survives as an explicitly secondary control for diagnosing one decision. Human takeover is now recorded: when the loop hands back, scanSession starts a listener that watches (never intercepts — the mirror image of the existing Ctrl+Click picker, which must suppress) every click and field change in the live window, resolves it to the UI5 control the human meant using the same outermost-actionable ancestor walk and the same classifyControl the picker already uses, registers it through the same objectRepository.upsert() path everything else goes through, and appends it to the run's own step log — so pressing "Carry on from here" resumes with the model able to see what the human did instead of repeating it. Each interaction is snapshotted inside the page at the moment it happens rather than looked up afterwards, because the commonest human action is the one that navigates away from the control it touched. 12 new regression tests (7 for the capture-side labelling and list-item rules against a synthetic UI5 stub, 5 for the new candidate rules including one asserting no control id can appear in a prompt and one asserting a repeat-action fallback names the control rather than interpolating an absent label) plus 3 new API guard tests for /api/discovery/run and /api/discovery/human-step. acceptanceProgress correctly stays 0/4 — none of this is proven live yet. DEFERRED on 4 Aug 2026 by owner decision, with the implementation removed from

main rather than left in place unused. The removal was deliberately surgical, not a wholesale revert of the 23 BL-047 commits: three changes inside them fix code shared with features that have nothing to do with discovery, and reverting those would have reintroduced real defects. Kept: ui5Inspector's rendered-only filter (the Fiori Launchpad keeps a previous app's whole component alive and hidden, so without it every manual capture and every Reconcile picks up dozens of stale invisible controls), its tooltip/icon label fallback and the sap.m list-item family being actionable (both improve manual capture directly), the HC-023 data-tab race fix (entirely unrelated, merely found during this work), Object reconciliation and the "Reconcile all" action (a standalone manual feature needing no discovery at all), and Global Search's unknown-process routing (an extension of BL-037). Removed: the discovery navigation policy, the live orchestration loop and its session, control-name derivation, the AI resolver seam and its Anthropic implementation, encrypted AI-provider credential storage and its Settings section, QueryReferenceDocument, the human-takeover recorder, FioriPlaywrightAdapter.attach(), getActivePage(), every /api/discovery/* and /api/settings/ai-provider route, the discovery panel in Control Object Repository, and the nine test files covering them. queryValidLineItemData.ts, which QueryReferenceDocument generalised without replacing, is untouched and still present. Work still uncommitted when the item was dropped — a disabled-control capture fix, and third-party draft Test/Dataset generation that was never tested — was first parked on a bl047-wip-archive branch and then, on 4 Aug 2026, discarded outright by owner decision; that branch no longer exists and those ~300 lines are gone. What survives is everything BL-047 ever committed: all 23 commits up to 810821a remain in history in full, nothing was rewritten or force-pushed, so resuming the item would start from real committed work rather than from nothing.

Evidence: Workspace owner requirement, 31 Jul 2026 · docs/ui-ux/AUTONOMOUS_TEST_AUTHORING_DESIGN.md · object-repository.db controls table (53/53 rows with NULL created_at) · packages/core/src/domain/objectRepository.ts upsert() · packages/studio-server/src/server.ts POST /api/objects/:appId/reconcile · packages/studio-web/src/components/ObjectBrowser.tsx Reconcile all action · packages/studio-web/src/components/GlobalSearchPanel.tsx unknown-process routing · packages/engine/src/modules/queryReferenceDocument.ts · packages/engine/src/discoveryNavigation.ts · packages/engine/src/discoveryRegistration.ts · packages/studio-server/src/discoverySession.ts · packages/adapter-fiori/src/fioriAdapter.ts FioriPlaywrightAdapter.attach() · packages/studio-web/src/components/ObjectScanner.tsx Autonomous discovery panel · packages/core/src/aiCredentials.ts · packages/engine/src/aiResolver.ts · packages/studio-server/src/anthropicResolver.ts · packages/studio-web/src/components/SettingsPanel.tsx AI provider section · regression/api/objects.test.js · regression/api/discovery.test.js · regression/api/ai-provider.test.js · regression/ui/object-repository.test.js · regression/ui/global-search.test.js · regression/ui/ai-provider-settings.test.js · regression/query-reference-document.test.js · regression/discovery-navigation.test.js · regression/discovery-registration.test.js · regression/ai-credentials-fallback.test.js · regression/anthropic-resolver.test.js · packages/adapter-fiori/src/ui5Inspector.ts · regression/ui5-inspector-visibility.test.js
Last reconciled: 2026-08-04

RECORDED FAILURE LEDGER

4 Defects and issues fixed

The product test ledger preserves failures even after remediation. All 59 recorded failures are remediated and linked to a subsequent verifying run.

ID / status	Area / mode	Observed failure	Resolution evidence
FL-0059 Remediated	Execution Center Unit / Integration · Isolated	**sort by durationMs and status in both directions (BL-035 AC2)** Expected values to be strictly deep-equal: + actual - expected ['short', + '711260c9-e8a7-44ee-a9ad-d5e1a63b0029', + '882140a3-c4b2-11ee-a8c5-0242ac120002', + '072380f6-d5a5-33cc-b0be-d4e5f67a8901', + '901270d8-f7b8-55…	Verified by recorded run CORE-2026-08-10T17-25-35-707Z. Verified: CORE-2026-08-10T17-25-35-707Z
FL-0058 Remediated	Execution Center Unit / Integration · Isolated	**pagination returns the correct slice and total across pages (BL-035 AC2)** Expected values to be strictly equal: 10 !== 5	Verified by recorded run CORE-2026-08-10T17-25-35-707Z. Verified: CORE-2026-08-10T17-25-35-707Z
FL-0057 Remediated	Execution Center Unit / Integration · Isolated	**every AC1 filter narrows the result set independently** Expected values to be strictly deep-equal: + actual - expected [+ '312280e7-e6a6-44dd-c1cf-f7e3c85d0241', 'run-b']	Verified by recorded run CORE-2026-08-10T17-25-35-707Z. Verified: CORE-2026-08-10T17-25-35-707Z
FL-0056 Remediated	Execution Center Unit / Integration · Isolated	**record computes durationMs and list/get round-trip every new field (BL-035 AC1/AC3/AC4)** Expected values to be strictly equal: 6 !== 1	Verified by recorded run CORE-2026-08-10T17-25-35-707Z. Verified: CORE-2026-08-10T17-25-35-707Z
FL-0055 Remediated	Execution Center Unit / Integration · Isolated	**sort by durationMs and status in both directions (BL-035 AC2)** Expected values to be strictly deep-equal: + actual - expected ['short', + '711260c9-e8a7-44ee-a9ad-d5e1a63b0029', + '882140a3-c4b2-11ee-a8c5-0242ac120002', + '072380f6-d5a5-33cc-b0be-d4e5f67a8901', + '901270d8-f7b8-55…	Verified by recorded run CORE-2026-08-10T17-25-35-707Z. Verified: CORE-2026-08-10T17-25-35-707Z
FL-0054 Remediated	Execution Center Unit / Integration · Isolated	**pagination returns the correct slice and total across pages (BL-035 AC2)** Expected values to be strictly equal: 10 !== 5	Verified by recorded run CORE-2026-08-10T17-25-35-707Z. Verified: CORE-2026-08-10T17-25-35-707Z
FL-0053 Remediated	Execution Center Unit / Integration · Isolated	**every AC1 filter narrows the result set independently** Expected values to be strictly deep-equal: + actual - expected [+ '312280e7-e6a6-44dd-c1cf-f7e3c85d0241', 'run-b']	Verified by recorded run CORE-2026-08-10T17-25-35-707Z. Verified: CORE-2026-08-10T17-25-35-707Z

ID / status	Area / mode	Observed failure	Resolution evidence
FL-0052 Remediated	Execution Center Unit / Integration · Isolated	**record computes durationMs and list/get round-trip every new field (BL-035 AC1/AC3/AC4)** Expected values to be strictly equal: 6 !== 1	Verified by recorded run CORE-2026-08-10T17-25-35-707Z. Verified: CORE-2026-08-10T17-25-35-707Z
FL-0051 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ + name: 'encrypted AI provider API key persists securely across processes, separate from SAP credentials', + source: 'regression/ai-cr...	Verified by recorded run CORE-2026-08-10T03-52-49-895Z. Verified: CORE-2026-08-10T03-52-49-895Z
FL-0050 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'encrypted AI provider API key persists securely across processes, separate from SAP credentials', source: 'regression/ai-creden...	Verified by recorded run CORE-2026-08-04T05-05-28-893Z. Verified: CORE-2026-08-04T05-05-28-893Z
FL-0049 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'encrypted AI provider API key persists securely across processes, separate from SAP credentials', source: 'regression/ai-creden...	Verified by recorded run CORE-2026-08-04T04-26-20-269Z. Verified: CORE-2026-08-04T04-26-20-269Z
FL-0048 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'encrypted AI provider API key persists securely across processes, separate from SAP credentials', source: 'regression/ai-creden...	Verified by recorded run CORE-2026-08-04T03-22-09-170Z. Verified: CORE-2026-08-04T03-22-09-170Z
FL-0047 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'encrypted AI provider API key persists securely across processes, separate from SAP credentials', source: 'regression/ai-creden...	Verified by recorded run CORE-2026-08-04T02-31-22-900Z. Verified: CORE-2026-08-04T02-31-22-900Z

ID / status	Area / mode	Observed failure	Resolution evidence
FL-0046 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'encrypted AI provider API key persists securely across processes, separate from SAP credentials', source: 'regression/ai-creden…	Verified by recorded run CORE-2026-08-03T15-59-37-278Z. Verified: CORE-2026-08-03T15-59-37-278Z
FL-0045 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'encrypted AI provider API key persists securely across processes, separate from SAP credentials', source: 'regression/ai-creden…	Verified by recorded run CORE-2026-08-03T15-44-03-651Z. Verified: CORE-2026-08-03T15-44-03-651Z
FL-0044 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'encrypted AI provider API key persists securely across processes, separate from SAP credentials', source: 'regression/ai-creden…	Verified by recorded run CORE-2026-08-03T14-57-14-784Z. Verified: CORE-2026-08-03T14-57-14-784Z
FL-0043 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'encrypted AI provider API key persists securely across processes, separate from SAP credentials', source: 'regression/ai-creden…	Verified by recorded run CORE-2026-08-03T14-32-51-081Z. Verified: CORE-2026-08-03T14-32-51-081Z
FL-0042 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ - name: 'encrypted AI provider API key persists securely across processes, separate from SAP credentials', - source: 'regression/ai-cr…	Verified by recorded run CORE-2026-08-03T14-11-54-573Z. Verified: CORE-2026-08-03T14-11-54-573Z
FL-0041 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'GET /api/audit/runs supports every AC1 filter and reports X-Total-Count for pagination (BL-035 AC1/AC2)', source: 'regression/a…	Verified by recorded run CORE-2026-08-03T10-55-09-805Z. Verified: CORE-2026-08-03T10-55-09-805Z

ID / status	Area / mode	Observed failure	Resolution evidence
FL-0040 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'GET /api/audit/runs supports every AC1 filter and reports X-Total-Count for pagination (BL-035 AC1/AC2)', source: 'regression/a...	Verified by recorded run CORE-2026-08-03T10-10-17-314Z. Verified: CORE-2026-08-03T10-10-17-314Z
FL-0039 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'GET /api/audit/runs supports every AC1 filter and reports X-Total-Count for pagination (BL-035 AC1/AC2)', source: 'regression/a...	Verified by recorded run CORE-2026-08-03T09-22-05-561Z. Verified: CORE-2026-08-03T09-22-05-561Z
FL-0038 Remediated	Test Data Headless UI · Isolated	**Dataset Library search actually filters by file name and does not silently look like a load failure (HC-023)** expected the Dataset Library to load at least one real dataset, got "0 of 0 datasets"	Verified by recorded run ISOLATED-UI-2026-08-03T08-55-47-121Z. Verified: ISOLATED-UI-2026-08-03T08-55-47-121Z
FL-0037 Remediated	Test Data Headless UI · Isolated	**Dataset Library search actually filters by file name and does not silently look like a load failure (HC-023)** expected the Dataset Library to load at least one real dataset, got "0 of 0 datasets"	Verified by recorded run ISOLATED-UI-2026-08-03T08-55-47-121Z. Verified: ISOLATED-UI-2026-08-03T08-55-47-121Z
FL-0036 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'GET /api/audit/runs supports every AC1 filter and reports X-Total-Count for pagination (BL-035 AC1/AC2)', source: 'regression/a...	Verified by recorded run CORE-2026-08-03T08-35-27-650Z. Verified: CORE-2026-08-03T08-35-27-650Z
FL-0035 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'GET /api/audit/runs supports every AC1 filter and reports X-Total-Count for pagination (BL-035 AC1/AC2)', source: 'regression/a...	Verified by recorded run CORE-2026-07-31T07-25-59-536Z. Verified: CORE-2026-07-31T07-25-59-536Z

ID / status	Area / mode	Observed failure	Resolution evidence
FL-0034 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'GET /api/audit/runs supports every AC1 filter and reports X-Total-Count for pagination (BL-035 AC1/AC2)', source: 'regression/a...	Verified by recorded run CORE-2026-07-31T05-35-48-306Z. Verified: CORE-2026-07-31T05-35-48-306Z
FL-0033 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'GET /api/audit/runs supports every AC1 filter and reports X-Total-Count for pagination (BL-035 AC1/AC2)', source: 'regression/a...	Verified by recorded run CORE-2026-07-30T15-42-35-981Z. Verified: CORE-2026-07-30T15-42-35-981Z
FL-0032 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'GET /api/audit/runs supports every AC1 filter and reports X-Total-Count for pagination (BL-035 AC1/AC2)', source: 'regression/a...	Verified by recorded run CORE-2026-07-30T14-43-56-554Z. Verified: CORE-2026-07-30T14-43-56-554Z
FL-0031 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ - name: 'GET /api/audit/runs supports every AC1 filter and reports X-Total-Count for pagination (BL-035 AC1/AC2)', - source: 'regressi...	Verified by recorded run CORE-2026-07-30T14-09-14-995Z. Verified: CORE-2026-07-30T14-09-14-995Z
FL-0030 Remediated	Execution Center API · Isolated	**a failed Chain reports a focused diagnosis with an exact correction link (BL-032 AC1/AC3)** Expected values to be strictly equal: 'passed' !== 'failed'	Verified by recorded run ISOLATED-API-2026-07-30T13-26-11-045Z. Verified: ISOLATED-API-2026-07-30T13-26-11-045Z
FL-0029 Remediated	Test Data Headless UI · Isolated	**unsaved new dataset is protected from shell navigation** locator.waitFor: Error: strict mode violation: getByRole('heading', { name: 'Test Data' }) resolved to 2 elements: 1) <h1 class="sr-only">Test Data</h1> aka getByRole('heading', { name: 'Test Data', exact: true }) 2) <h...	Verified by recorded run ISOLATED-UI-2026-07-30T12-35-09-886Z. Verified: ISOLATED-UI-2026-07-30T12-35-09-886Z

ID / status	Area / mode	Observed failure	Resolution evidence
FL-0028 Remediated	Application Shell Headless UI · Isolated	**browser back and forward restore route-selected artifacts** locator.click: Timeout 30000ms exceeded. Call log: - waiting for getByRole('listbox', { name: 'Open dataset' }).getByRole('option', { name: 'regression-sample.csv', exact: true })	Verified by recorded run ISOLATED-UI-2026-07-30T12-35-09-886Z. Verified: ISOLATED-UI-2026-07-30T12-35-09-886Z
FL-0027 Remediated	Compose Headless UI · Isolated	**artifact detail routes restore Compose, Data, Process, Object, and Audit context after refresh** locator.inputValue: Timeout 30000ms exceeded. Call log: - waiting for locator('tbody tr').first().locator('input').first()	Verified by recorded run ISOLATED-UI-2026-07-30T12-35-09-886Z. Verified: ISOLATED-UI-2026-07-30T12-35-09-886Z
FL-0026 Remediated	Test Data Headless UI · Isolated	**Data: create a dataset, add a row, save, reload, reopen (required Data positive)** locator.fill: Timeout 30000ms exceeded. Call log: - waiting for locator('tbody tr').first().locator('input').first()	Verified by recorded run ISOLATED-UI-2026-07-30T12-35-09-886Z. Verified: ISOLATED-UI-2026-07-30T12-35-09-886Z
FL-0025 Remediated	Test Data Headless UI · Isolated	**unsaved new dataset is protected from shell navigation** locator.fill: Timeout 30000ms exceeded. Call log: - waiting for getByLabel('New dataset file name')	Verified by recorded run ISOLATED-UI-2026-07-30T12-35-09-886Z. Verified: ISOLATED-UI-2026-07-30T12-35-09-886Z
FL-0024 Remediated	Application Shell Headless UI · Isolated	**browser back and forward restore route-selected artifacts** locator.click: Timeout 30000ms exceeded. Call log: - waiting for getByRole('button', { name: 'Open dataset' })	Verified by recorded run ISOLATED-UI-2026-07-30T12-35-09-886Z. Verified: ISOLATED-UI-2026-07-30T12-35-09-886Z
FL-0023 Remediated	Compose Headless UI · Isolated	**artifact detail routes restore Compose, Data, Process, Object, and Audit context after refresh** locator.inputValue: Timeout 30000ms exceeded. Call log: - waiting for locator('tbody tr').first().locator('input').first()	Verified by recorded run ISOLATED-UI-2026-07-30T12-35-09-886Z. Verified: ISOLATED-UI-2026-07-30T12-35-09-886Z
FL-0022 Remediated	Test Data Headless UI · Isolated	**Data: relate header and child CSVs, preview counts, and persist the relationship** locator.fill: Timeout 30000ms exceeded. Call log: - waiting for getByLabel('Relationship name')	Verified by recorded run ISOLATED-UI-2026-07-30T12-16-12-125Z. Verified: ISOLATED-UI-2026-07-30T12-16-12-125Z
FL-0021 Remediated	Test Data Headless UI · Isolated	**Data: author and preview nested JSON transactions before save** locator.selectOption: Timeout 30000ms exceeded. Call log: - waiting for getByLabel('New dataset format')	Verified by recorded run ISOLATED-UI-2026-07-30T12-16-12-125Z. Verified: ISOLATED-UI-2026-07-30T12-16-12-125Z

ID / status	Area / mode	Observed failure	Resolution evidence
FL-0020 Remediated	Test Data Headless UI · Isolated	**Data: create a dataset, add a row, save, reload, reopen (required Data positive)** locator.fill: Timeout 30000ms exceeded. Call log: - waiting for locator('input[placeholder="my-new-dataset"]')	Verified by recorded run ISOLATED-UI-2026-07-30T12-35-09-886Z. Verified: ISOLATED-UI-2026-07-30T12-35-09-886Z
FL-0019 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'GET /api/data lists known fixtures', source: 'regression/api/data.test.js' }, ... { - name: 'PUT /api/objects/:appId/:name save...	Verified by recorded run CORE-2026-07-30T11-28-54-635Z. Verified: CORE-2026-07-30T11-28-54-635Z
FL-0018 Remediated	Object Repository Headless UI · Isolated	**publishing is blocked by required parameters, missing objects and output collisions** locator.click: Timeout 30000ms exceeded. Call log: - waiting for getByRole('button', { name: 'Save step' }) - locator resolved to <button class="primary">Save step</button> - attempting click action 2 × waiting for elem...	Verified by recorded run ISOLATED-UI-2026-07-30T10-49-58-013Z. Verified: ISOLATED-UI-2026-07-30T10-49-58-013Z
FL-0017 Remediated	Execution Center Unit / Integration · Isolated	**Test Operations catalogue inventory and status fields come only from repository tests and recorded runs** Expected values to be strictly deep-equal: + actual - expected ... Skipped lines [{ name: 'GET /api/data lists known fixtures', source: 'regression/api/data.test.js' }, ... { - name: 'Compose: contextual capture opens ...	Verified by recorded run CORE-2026-07-30T10-39-34-975Z. Verified: CORE-2026-07-30T10-39-34-975Z
FL-0016 Remediated	Process Suites Live UI · Non-production SAP	**live: Run tab read-only Batch runs two independent SAP smoke groups** expected both groups to pass, got: "✓ All 2 process iterations passed exit code 0 2 of 2 process iterations completed 100%"	Reconciled the UI assertion with hierarchical Batch completion wording and reverified both live smoke groups. Verified: T1.3.1-LIVE-UI-R2
FL-0015 Remediated	Test Data Live UI · Non-production SAP	**Run tab: a preflight data finding opens the exact dataset route** locator.click: Timeout 30000ms exceeded. Call log: - waiting for locator('li:has-text("route-mapped.json") button:has-text("+ Add")')	Aligned the correction journey with the route-aware dataset selection flow and reverified the exact dataset route. Verified: T1.3.1-ISOLATED-UI-R4
FL-0014 Remediated	Compose API · Synthetic non-production	**GET /api/testcases/:file returns the parsed test case** Expected values to be strictly equal: 404 !== 200	Corrected isolated Test fixture provisioning so the detail endpoint returns the persisted Test. Verified: T1.3.1-ISOLATED-API-R2
FL-0013 Remediated	Compose API · Synthetic non-production	**GET /api/testcases lists known fixtures** expected create-po.json in the list	Corrected isolated Test fixture discovery so known Tests appear in the list. Verified: T1.3.1-ISOLATED-API-R2

ID / status	Area / mode	Observed failure	Resolution evidence
FL-0012 Remediated	Process Suites API · Synthetic non-production	**GET /api/groups/:file returns the parsed group** Expected values to be strictly equal: + actual - expected + 'Synthetic Process' - 'Create PO - GR - Invoice'	Corrected isolated Business Process fixture identity and response expectations. Verified: T1.3.1-ISOLATED-API-R2
FL-0011 Remediated	Process Suites API · Synthetic non-production	**GET /api/groups lists known fixtures** The expression evaluated to a falsy value: assert.ok(body.includes('o2c-e2e.json'))	Corrected isolated Business Process fixture discovery for the O2C process. Verified: T1.3.1-ISOLATED-API-R2
FL-0010 Remediated	Test Data API · Synthetic non-production	**GET /api/data/:file parses an existing CSV** The expression evaluated to a falsy value: assert.ok(body.headers.includes('supplier'))	Corrected isolated CSV fixture provisioning and header expectations. Verified: T1.3.1-ISOLATED-API-R2
FL-0009 Remediated	Test Data API · Synthetic non-production	**GET /api/data lists known fixtures** The expression evaluated to a falsy value: assert.ok(body.includes('p2p-e2e.csv'))	Corrected isolated data fixture discovery for P2P data. Verified: T1.3.1-ISOLATED-API-R2
FL-0008 Remediated	Process Suites Headless UI · Synthetic non-production	**Run tab: Pack Processes completes a synthetic multi-group Batch** expected two Batch groups to pass, got: "✓ All 2 process iterations passed exit code 2 of 2 process iterations completed 100%"	Reconciled Pack · Processes completion output with the two-member hierarchical result. Verified: T1.3.1-ISOLATED-UI-R3
FL-0007 Remediated	Execution Center Headless UI · Synthetic non-production	**Run tab: Pack Tests completes a synthetic independent Suite** Run did not complete within 30000ms	Stabilized synthetic Pack · Tests execution completion and polling within the supported run lifecycle. Verified: T1.3.1-ISOLATED-UI-R3
FL-0006 Remediated	Process Suites Headless UI · Synthetic non-production	**Run tab: Business Process completes a synthetic multi-stage Chain** Run did not complete within 30000ms	Stabilized synthetic multi-stage Business Process completion and polling. Verified: T1.3.1-ISOLATED-UI-R3
FL-0005 Remediated	Process Suites Headless UI · Synthetic non-production	**Run tab: Pack Processes completes a synthetic multi-group Batch** [object Object]	Corrected the synthetic Pack · Processes execution path and verified two independent members. Verified: T1.3.1-ISOLATED-UI-R3
FL-0004 Remediated	Execution Center Headless UI · Synthetic non-production	**Run tab: Batch mode runs "Cleanup Drafts" to completion (execution opt-in)** [object Object]	Corrected the synthetic Batch compatibility execution path and completion state. Verified: T1.3.1-ISOLATED-UI-R2
FL-0003 Remediated	Execution Center Headless UI · Synthetic non-production	**Run tab: Pack Tests completes a synthetic independent Suite** [object Object]	Corrected the synthetic Pack · Tests execution path and independent member results. Verified: T1.3.1-ISOLATED-UI-R3
FL-0002 Remediated	Process Suites Headless UI · Synthetic non-production	**Run tab: Business Process completes a synthetic multi-stage Chain** [object Object]	Corrected the synthetic Business Process execution path and ordered stage results. Verified: T1.3.1-ISOLATED-UI-R3

ID / status	Area / mode	Observed failure	Resolution evidence
FL-0001 Remediated	Automation Overview Headless UI · Synthetic non-production	**Canvas First Overview presents the approved shell and real workspace data** [object Object]	Aligned the Overview fixture/API state with the approved Canvas First metrics and shell. Verified: T1.3.1-ISOLATED-UI-R2

Open defect count: zero in the recorded ledger. This statement is limited to the repository-backed failure ledger as of 30 July 2026; newly reported production observations must still be logged and triaged.

REMAINING ACCEPTANCE SCOPE

5 Items requiring remediation or future delivery

These are product gaps or intentionally deferred capabilities, not unverified claims of production defects.

ID / status	Item	Criteria still requiring work	Recommended remediation
BL-039 Deferred	**Add multi-user roles and capabilities** Platform, All workspaces · P3	• Implementation starts after a confirmed multi-user requirement. • Server protects every mutation, execution, artifact and administration action. • Role mapping covers manager, consultant, engineer, tester, analyst and auditor.	Keep deferred until multi-user requirements and role ownership are explicitly approved.
BL-040 Deferred	**Add scheduling, notification and controlled parallelism** Execution Center, Automation Overview · P3	• Schedules capture immutable scope, target and owner. • Notifications link to stable run records without exposing secrets. • Parallel work respects target capacity, data isolation and cancellation. • Concurrency limits and queue health are observable.	Keep deferred until schedules, notifications, target capacity and concurrency governance are designed.
BL-045 Not started	**Improve Test Data grid row readability** Test Data · P3	• Column values are legible without expanding every cell. • Dense per-cell controls (edit/expand icons) remain available but visually secondary. • Change does not regress the existing nested-JSON and relational CSV editing flows.	Refine and schedule the remaining acceptance criteria.
BL-046 Not started	**Group and organize Audit and Evidence at scale** Audit & Evidence · P3	• Long result sets are grouped (e.g. by date or App ID) or virtualized instead of one continuous scroll. • Existing filters, sort, pagination and lineage links keep working unchanged. • Behavior is verified against a realistically large synthetic run set, not just a handful of fixtures.	Refine and schedule the remaining acceptance criteria.

ID / status	Item	Criteria still requiring work	Recommended remediation
BL-047 Deferred	**Natural-language-driven test authoring: reuse what's known, autonomously discover what isn't** Control Object Repository, Compose, Test Data, Execution Center · P0	• A natural-language process name routes to an existing App ID/Test when real, captured Objects already exist for it — no discovery attempted, no behavior change from today's manual workflow. • When the App ID/process is unknown, the engine launches the configured SAP tenant, locates a previously-created business document of the matching type as a live reference, and extracts reusable master data from it, without a human selecting either. • The engine drives the live process end-to-end using that master data, capturing every control it actually interacts with into the Object Repository through the same upsert path (real createdAt/updatedAt, real duplicate/unstable-id detection) a manual capture uses — never a fabricated or scripted control definition. • A Test is composed from the exact discovered interaction sequence and a Test Data instance is generated from the extracted master data and attached to it, both presented in a review screen for the owner to accept, edit or reject before either is treated as final; the same design generalises to other processes (e.g. Create Production Order) without process-specific code.	Refine and schedule the remaining acceptance criteria.

DELIVERY GUIDANCE

6 Verification evidence and recommended next sequence

Evidence base used

- Authoritative HTML Product Backlog Tracker.
- Repository-generated 234-test operations catalogue.
- Recorded execution and failure history.
- Core, API, UI, accessibility, secret-scan and live-run documentation.
- Current production build source and anonymized workspace captures.

Recommended next implementation sequence

1. BL-018 and BL-019: complete Overview deep links, alerts and governed analytics.
2. BL-022 through BL-025: finish object and data governance foundations.
3. BL-031, BL-032 and BL-035: close execution hierarchy, diagnosis and audit traversal.
4. BL-037: add global search only after dependency metadata is reliable.
5. Retain BL-039 and BL-040 as deferred until multi-user and scheduling scope is approved.

Change-control recommendation. Update the HTML tracker and regenerate this report after each accepted backlog tranche. Record every failed automated case in the durable ledger even when it is subsequently remediated.